KIỂM THỬ

BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC KINH TẾ
KỸ THUẬT CÔNG NGHIỆP

KHOA: CÔNG NGHỆ
THÔNG TIN

BÁO CÁO ĐỒ ÁN
WEBSITE ĐĂNG KÝ HIẾN MÁU NHÂN ĐẠO

Họ và tên sinh viên:
Nguyễn Minh Ngọc
Đỗ Xuân Trường

HÀ NỘI - 2025

KIỂM THỬ

WEBSITE ĐĂNG KÝ HIẾN MÁU NHÂN ĐẠO

Mục lục

LỜI MỞ ĐẦU 1
1. GIỚI THIỆU CHUNG 2
2. KHẢO SÁT DỰ ÁN 3
2.1. Mô tả dự án 3
2.2. Mục tiêu của giai đoạn khảo sát 3
2.3. Khảo sát sơ bộ 4
2.3.1. Mục tiêu 4
2.3.2. Phương pháp khảo sát 4
2.3.3. Nội dung khảo sát sơ bộ 5
2.4. Khảo sát chi tiết 5
2.4.1. Mục tiêu 5
2.4.2. Phương pháp khảo sát 5
2.4.3. Nội dung khảo sát chi tiết 6
2.4.3.1. Chức năng người dùng 6
2.4.3.2. Chức năng quản trị viên 6
2.4.3.3. Chức năng bệnh viện 7
2.4.3.4. Chức năng bác sĩ 7
2.4.3.5. Chức năng nhân viên y tế 7
2.4.3.6. Dữ liệu cần quản lý 7
2.4.3.7. Yêu cầu phi chức năng 9
2.5. Các vấn đề trọng tâm cần giải quyết 9
3. PHÂN TÍCH HỆ THỐNG 10
3.1. Xác định yêu cầu hệ thống 10
3.2. Mô hình 12
3.2.1. Sơ đồ phân rã chức năng 13
3.2.2. Sơ đồ luồng dữ liệu mức đỉnh 13
3.2.3. Sơ đồ luồng dữ liệu mức dưới đỉnh 14
3.2.4. Sơ đồ luồng dữ liệu quản lý người dùng 15
3.2.5. Sơ đồ luồng dữ liệu quản lý sự kiện hiến máu 16
3.2.6. Sơ đồ luồng dữ liệu đăng ký hiến máu 17

3.2.7. Sơ đồ luồng dữ liệu sàng lọc sức khỏe 18
3.2.8. Sơ đồ luồng dữ liệu lưu trữ lịch sử hiến máu 19
3.2.9. Sơ đồ luồng dữ liệu quản lý tin tức & thông báo 20
3.2.10. Sơ đồ luồng dữ liệu cấu hình hệ thống 21
3.2.11. Sơ đồ luồng dữ liệu báo cáo & thống kê 22
3.2.12. Sơ đồ thực thể quan hệ 23
3.3. Phân tích cơ sở dữ liệu 24
3.3.1. Danh sách các bảng dữ liệu chính 24
3.3.2. Chi tiết 25
a. Bảng Roles 25
b. Bảng Users 25
c. Bảng BloodTypes 26
d. Bảng BloodCompatibility 26
e. Bảng Locations 26
f. Bảng BloodDonationEvents 27
g. Bảng DonationRegistrations 28
h. Bảng HealthScreening 28
i. Bảng DonationHistory 29
j. Bảng NewsCategories 29
k. Bảng News 30
l. Bảng Notifications 30
m. Bảng Settings 31
n. Bảng ContactMessages 31
3.3.3. Quan hệ giữa các bảng 32
3.3.4. Ràng buộc dữ liệu 33
4. THIẾT KẾ 37
4.1. Mục tiêu thiết kế: 37
4.2. Mô tả kiến trúc hệ thống 37
4.3. Các quyết định thiết kế chính 39
4.3.1. Sử dụng mô hình MVC 39
4.3.2. Entity Framework Core cho truy cập dữ liệu 42
4.3.3. Sử dụng DTOs và ViewModels 42
4.3.4. Dependency Injection 43

4.3.5. Xác thực và phân quyền 44
4.4. Các mẫu thiết kế được áp dụng 45
4.5. Luồng dữ liệu 45
5. THỰC HIỆN 46
5.1. Công nghệ và công cụ sử dụng 46
5.2. Hiện thực hóa các lớp kiến trúc 46
5.3. Tích hợp luồng hoạt động 48
5.3.1. Tổng quan 48
5.3.2. Các bước trong quy trình hiến máu 48
5.3.3. Xử lý trường hợp đặc biệt 53
5.3.4. Bảng phân quyền 54
6. KIỂM THỬ 56
6.1. Tổng quan 56
6.1.1. Thông tin chung 56
6.1.2. Lý do kiểm thử 57
6.1.3. Vai trò kiểm thử 57
6.1.4. Các vấn đề phát sinh nếu không kiểm thử 58
6.2. Mục tiêu kiểm thử 59
6.3. Phương pháp kiểm thử 60
6.3.1. Loại kiểm thử 60
6.3.2. Công cụ kiểm thử 60
6.3.3. Môi trường kiểm thử 60
6.4. Test Case 61
6.4.1. Đăng ký tài khoản 61
6.4.2. Đăng nhập 62
6.4.3. Đăng ký sự kiện 63
6.4.4. Check-in sự kiện 64
6.4.5. Sàng lọc sức khoẻ 65
6.4.6. Hiến máu 66
6.4.7. Hoàn tất 67
6.5. Kiểm thử bảo mật và RBAC 68
6.6. Thực hiện kiểm thử 69
6.7. Kết quả kiểm thử 70

6.8. Thách thức và giải pháp 70
6.9. Kết luận 70
7. TRIỂN KHAI 71
7.1. Mục tiêu triển khai 71
7.2. Quản Lý Mã Nguồn & Quy Trình CI/CD 71
7.3. Chuẩn Bị Môi Trường Phát Triển 71
7.4. Khởi Tạo & Cập Nhật Cơ Sở Dữ Liệu 72
7.5. Build, Kiểm Thử & Đóng Gói 72
7.6. Cấu Hình Triển Khai 72
7.7. Bảo Mật & Hiệu Suất 72
7.8. Quản Lý & Bảo Trì 73

LỜI MỞ ĐẦU

Hiến máu nhân đạo là một hành động nhân văn cao đẹp, góp phần cứu sống hàng triệu bệnh nhân mỗi năm. Tuy nhiên, tại Việt Nam cũng như nhiều quốc gia khác, các hoạt động liên quan đến hiến máu vẫn còn gặp nhiều thách thức: nguồn máu dự trữ thường xuyên thiếu hụt, công tác vận động hiến máu chủ yếu thông qua các kênh truyền thống như phát tờ rơi, gọi điện thoại hoặc tổ chức các sự kiện tập trung, dẫn đến quy trình kết nối giữa người cần máu và người hiến máu còn rời rạc, thiếu đồng bộ và tốn nhiều nguồn lực.

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ và quá trình chuyển đổi số đang diễn ra trong nhiều lĩnh vực, việc ứng dụng công nghệ vào công tác tổ chức, quản lý và vận động hiến máu là yêu cầu rất cần thiết. Công nghệ thông tin không chỉ giúp nâng cao hiệu quả, minh bạch hóa quy trình mà còn tạo sự tiện lợi, nhanh chóng cho cả người hiến máu và các đơn vị tiếp nhận.

Xuất phát từ thực tiễn đó, em chọn đề tài "Thiết kế website đăng ký hiến máu nhân đạo" với mục tiêu xây dựng một nền tảng trực tuyến thân thiện, cho phép người dùng dễ dàng đăng ký hiến máu, theo dõi lịch sử hiến máu của bản thân và tiếp cận các thông tin về những đợt hiến máu mới do các tổ chức phát động. Hệ thống còn hỗ trợ các tổ chức có thể quản lý thông tin người hiến máu, thống kê dữ liệu phục vụ công tác điều phối và lập kế hoạch.

Đối tượng sử dụng chính của hệ thống là các cá nhân có nhu cầu hiến máu, các tổ chức tiếp nhận máu (như bệnh viện, hội chữ thập đỏ), cùng đội ngũ tình nguyện viên. Phạm vi đề tài tập trung phát triển các chức năng đăng ký tài khoản, đăng ký hiến máu, quản lý thông tin người dùng và thống kê hiến máu. Website được xây dựng trên nền tảng lập trình C#, cơ sở dữ liệu SQL Server và giao diện HTML/CSS.

1. GIỚI THIỆU CHUNG

Ngày nay, công nghệ thông tin đã trở thành một phần không thể thiếu trong mọi lĩnh vực của đời sống, đặc biệt là trong ngành y tế và các hoạt động nhân đạo. Việc ứng dụng các giải pháp công nghệ vào công tác vận động, quản lý và kết nối người hiến máu đang trở thành nhu cầu cấp thiết. Tại Việt Nam, mặc dù công nghệ thông tin mới phát triển mạnh trong những năm gần đây, nhưng đã chứng minh được vai trò quan trọng trong việc nâng cao hiệu quả các chương trình hiến máu nhân đạo.

Hiện nay, các bệnh viện và trung tâm huyết học thường xuyên đối mặt với tình trạng thiếu máu dự trữ, đặc biệt là các nhóm máu hiếm. Công tác vận động hiến máu chủ yếu vẫn được thực hiện theo phương thức truyền thống như phát tờ rơi, gọi điện thoại hoặc qua các chiến dịch hiến máu tập trung. Cách làm này tồn tại nhiều hạn chế như tốn nhiều thời gian, nhân lực, khó khăn trong việc quản lý thông tin người hiến máu và không chủ động được trong các trường hợp cần máu gấp.

Trước thực trạng đó, việc xây dựng một website đăng ký hiến máu nhân đạo trực tuyến là giải pháp tối ưu, giúp kết nối nhanh chóng giữa người có nhu cầu hiến máu với các đơn vị tiếp nhận. Hệ thống này sẽ giúp tự động hóa quy trình đăng ký, quản lý thông tin người hiến máu một cách khoa học, đồng thời tạo điều kiện thuận lợi để kịp thời đáp ứng nhu cầu máu trong các trường hợp khẩn cấp. Qua đó, góp phần nâng cao hiệu quả của các chương trình hiến máu nhân đạo, tiết kiệm chi phí và thời gian cho cả người hiến máu lẫn các đơn vị y tế.

2. KHẢO SÁT DỰ ÁN

2.1. Mô tả dự án

Dự án xây dựng một website hỗ trợ đăng ký, quản lý và tổ chức các hoạt động hiến máu nhân đạo một cách hiệu quả, hiện đại và tự động hóa. Hệ thống giúp kết nối người dân có nhu cầu hiến máu với các tổ chức, bệnh viện cần nguồn máu, đồng thời hỗ trợ đơn vị tổ chức quản lý thông tin sự kiện, người tham gia và thống kê dữ liệu phục vụ cho các chiến dịch lâu dài.

2.2. Mục tiêu của giai đoạn khảo sát

Giai đoạn khảo sát nhằm thu thập, phân tích các thông tin tổng quan và chi tiết liên quan đến hệ thống hiến máu nhân đạo. Từ đó, đưa ra các vấn đề trọng tâm cần giải quyết và đề xuất giải pháp khả thi, tạo tiền đề cho các bước phân tích thiết kế tiếp theo.

Khảo sát dự án được tiến hành qua 2 bước:

- Khảo sát sơ bộ: Tìm hiểu đặc điểm tổ chức, con người, và văn hóa hoạt động tại các đơn vị hiến máu tiêu biểu nhằm có cái nhìn tổng quát, xác định nhu cầu thực tế và đặc thù của từng nhóm đối tượng (người hiến máu, đơn vị tiếp nhận, quản trị hệ thống).
- Khảo sát chi tiết: Thu thập dữ liệu về chức năng cụ thể, quy trình nghiệp vụ, yêu cầu về thông tin, tính năng từ người dùng cuối và các bên liên quan để làm rõ phạm vi triển khai.

2.3. Khảo sát sơ bộ

2.3.1. Mục tiêu

- Xác định tổng quan về hiện trạng tổ chức và vận hành hệ thống hiến máu nhân đạo.
- Nắm được nhu cầu, kỳ vọng và khó khăn của các bên liên quan: người dân (người hiến máu), đơn vị tổ chức (bệnh viện, hội chữ thập đỏ), nhân viên y tế, quản trị viên hệ thống.
- Phát hiện các vấn đề, bất cập trong các quy trình hiện tại.
- Định hướng các chức năng và phạm vi triển khai của website đăng ký hiến máu.

2.3.2. Phương pháp khảo sát

- Phỏng vấn sơ bộ: Đã thực hiện phỏng vấn 2 cán bộ tổ chức tại Trung tâm Hiến máu nhân đạo TP.HCM và 1 nhân viên điều phối hiến máu của Hội Chữ thập đỏ để tìm hiểu nhu cầu thực tế cũng như khó khăn khi tiếp nhận thông tin từ người đăng ký truyền thống.
- Quan sát thực tế: Tham gia quan sát quy trình tổ chức một sự kiện hiến máu do Bệnh viện Truyền máu Huyết học tổ chức vào tháng 5/2025, ghi nhận các bước đăng ký hiến máu bằng giấy và tổng hợp các điểm bất cập trong việc nhập liệu, lưu trữ thông tin.
- Thu thập tài liệu: Nghiên cứu nội dung đăng tải trên website https://giotmauvang.org.vn/ và https://vienhuyethoc.vn/ cũng như thu thập mẫu báo cáo của Bệnh viện truyền máu trung ương về hiệu quả các đợt hiến máu trong năm 2024.
- Khảo sát online: Phát ra bảng khảo sát cho 100 người đã từng tham gia hiến máu tại TP.HCM, nhận được 62 phản hồi hợp lệ. 87% người được hỏi bày tỏ mong muốn có nền tảng đăng ký hiến máu trực tuyến, 74% mong muốn được thông báo tự động khi có đợt hiến máu gần nơi sinh sống.

2.3.3. Nội dung khảo sát sơ bộ

- Đối tượng sử dụng: Người dân (hiến máu), đơn vị tổ chức (bệnh viện, trung tâm huyết học, hội chữ thập đỏ), nhân viên y tế, quản trị viên hệ thống và tình nguyện viên hỗ trợ.
- Quy trình tổ chức hiến máu hiện tại: gồm các bước từ kêu gọi, tiếp nhận đăng ký, xác nhận thông tin người hiến máu, tổ chức lấy máu, ghi nhận và phản hồi kết quả tới người tham gia.
- Hình thức truyền thông, vận động: Thông qua mạng xã hội (Facebook, Zalo), website các đơn vị y tế, báo chí, lực lượng tình nguyện viên, truyền miệng…
- Các bất cập thường gặp: Khó khăn khi quản lý dữ liệu phân tán, người dân thiếu thông tin về các đợt hiến máu sắp tới, việc xác nhận người tham gia hoặc đối chiếu lịch sử hiến máu còn thủ công, thiếu thống kê cho công tác điều phối.

2.4. Khảo sát chi tiết

2.4.1. Mục tiêu

- Làm rõ yêu cầu nghiệp vụ cụ thể đối với từng chức năng trong hệ thống.
- Phân tích các quy trình công việc, luồng dữ liệu, vai trò của từng nhóm đối tượng sử dụng hệ thống.
- Xác định các yêu cầu phi chức năng cần thiết: bảo mật dữ liệu, hiệu suất xử lý, giao diện người dùng, khả năng mở rộng về lâu dài.

2.4.2. Phương pháp khảo sát

- Phỏng vấn nhóm nhỏ người dùng: Thực hiện phỏng vấn sâu 5 người đã từng hiến máu tại TP.HCM để tìm hiểu mong muốn về các chức năng như đăng ký tài khoản trực tuyến, nhận lịch nhắc hiến máu, tra cứu lịch sử hiến máu cá nhân. Kết quả cho thấy phần lớn người tham gia mong muốn quá trình đăng ký đơn giản, ít bước xác nhận phức tạp, nhận thông báo qua điện thoại khi có sự kiện gần nơi cư trú.
- Khảo sát online: Gửi bảng khảo sát trực tuyến cho 50 người có kinh nghiệm tham gia các chiến dịch hiến máu trong vòng 2 năm trở lại đây. Có 39 phản hồi hợp lệ, trong đó 82% mong muốn hệ thống có tính năng thông báo sự kiện, 75% ưu tiên

bảo mật thông tin cá nhân, 64% muốn có giao diện trực quan tích hợp trên điện thoại.

- Phỏng vấn chuyên gia/quản trị viên: Làm việc trực tiếp với 2 quản trị viên phụ trách tổ chức hiến máu tại Bệnh viện Truyền máu Huyết học TP.HCM và Hội Chữ thập đỏ, thu thập yêu cầu cụ thể về việc xuất báo cáo nhanh, quản lý sự kiện theo khu vực và nhóm máu.
- Mô phỏng quy trình: Triển khai mô phỏng thử quy trình tổ chức một sự kiện hiến máu qua website bằng việc tạo sự kiện giả lập, cho phép người dùng đăng ký, nhận thông báo và phản hồi. Quá trình này giúp xác định các bước truy cập, xác thực thông tin, trải nghiệm đăng ký, từ đó đề xuất các điểm cần tối ưu cho hệ thống thực tế.

2.4.3. Nội dung khảo sát chi tiết

2.4.3.1. Chức năng người dùng

- Đăng ký tài khoản / Đăng nhập hệ thống.
- Cập nhật thông tin cá nhân
- Xem sự kiện hiến máu
- Đăng ký hiến máu (theo đợt, theo lịch).
- Huỷ đăng ký
- Kiểm tra điều kiện hiến máu
- Check-in tại sự kiện
- Xem, quản lý lịch sử các lần hiến máu cá nhân.
- Nhận thông báo tự động khi có đợt hiến máu hoặc sự kiện phù hợp tại khu vực địa lý đã đăng ký.
- Gửi góp ý, phản hồi sau khi hoàn thành hiến máu.

2.4.3.2. Chức năng quản trị viên

- Đăng nhập hệ thống quản trị
- Tạo/sửa/xoá các sự kiện hiến máu.
- Quản lý người dùng
- Quản lý sự kiện
- Duyệt và quản lý danh sách người đăng ký tham gia cho từng sự kiện.
- Xuất báo cáo, thống kê người tham gia theo khu vực, độ tuổi, nhóm máu.

2.4.3.3. Chức năng bệnh viện

- Tạo/Sửa/Xoá sự kiện
- Theo dõi tiến độ sự kiện

2.4.3.4. Chức năng bác sĩ

- Xem danh sách người đến sự kiện
- Khám sàng lọc
- Đánh giá điều kiện sức khoẻ
- Hoàn tất hiến máu

2.4.3.5. Chức năng nhân viên y tế

- Check-in người hiến
- Thực hiện hiến máu
- Quan sát sau khi hiến máu
- Kết thúc phiên hiến máu

2.4.3.6. Dữ liệu cần quản lý

a, Thông tin cá nhân người dùng:

- Họ và tên.
- CMND/CCCD
- Nhóm máu.
- Địa chỉ.
- Số điện thoại.
- Ngày sinh.
- Giới tính.
- Email và trạng thái xác thực email.
- Trạng thái tài khoản (hoạt động hay không).
- Ngày hiến máu gần nhất.

b, Lịch sử hiến máu của từng cá nhân

- Ngày hiến máu.
- Địa điểm tổ chức.
- Kết quả xét nghiệm (cân nặng, chiều cao, huyết áp, nhịp tim, nhiệt độ, hemoglobin, lý do không đủ điều kiện nếu có).

- Phản hồi hoặc ghi chú.
- Lượng máu hiến.
- Trạng thái hiến máu (bắt đầu, hoàn thành, thất bại, v.v.).
- Ngày đủ điều kiện hiến máu tiếp theo.
- Thông tin giấy chứng nhận hiến máu.

c, Danh sách các sự kiện hiến máu (thông tin về thời gian, địa điểm, đơn vị tổ chức, lượng máu cần tuyển).

- Thời gian (ngày, giờ bắt đầu và kết thúc).
- Địa điểm tổ chức.
- Đơn vị tổ chức.
- Lượng máu cần tuyển và nhóm máu yêu cầu.
- Mô tả sự kiện.
- Số lượng người đăng ký và sức chứa tối đa.
- Trạng thái sự kiện (nháp, hoạt động, hoàn thành, hủy, v.v.).
- Hình ảnh minh họa sự kiện.

d, Thông tin liên lạc và mô tả chi tiết về từng điểm tổ chức hiến máu.

- Tên địa điểm.
- Địa chỉ.
- Số điện thoại liên lạc.
- Mô tả chi tiết về sự kiện tại địa điểm.
- Sức chứa và trạng thái hoạt động.

e, Tương thích nhóm máu: Thông tin về khả năng tương thích giữa các nhóm máu (nhóm máu cho và nhận).

f, Đăng ký hiến máu: Thông tin đăng ký của người dùng cho các sự kiện, bao gồm ngày đăng ký, trạng thái (đã đăng ký, xác nhận, check-in, hoàn thành, hủy, v.v.), thời gian check-in, thời gian hoàn thành, và lý do hủy nếu có.

g, Sàng lọc sức khỏe: Kết quả sàng lọc trước khi hiến máu, bao gồm thông tin y tế và người thực hiện sàng lọc.

h, Tin tức và danh mục tin tức: Tiêu đề, nội dung, tóm tắt, hình ảnh, danh mục, tác giả, số lượt xem, trạng thái xuất bản và ngày xuất bản.

i, Thông báo: Tiêu đề, nội dung, loại thông báo (đăng ký, nhắc nhở, thông tin, v.v.), trạng thái đọc và người nhận.

j, Tin nhắn liên hệ: Thông tin từ người gửi (họ tên, email, số điện thoại, chủ đề, nội dung), trạng thái xử lý và người xử lý.

2.4.3.7. Yêu cầu phi chức năng

- Giao diện dễ sử dụng, thân thiện với người dùng phổ thông.
- Bảo mật thông tin cá nhân.
- Hệ thống hoạt động tốt trên thiết bị di động.
- Tốc độ truy cập nhanh, dễ mở rộng trong tương lai.

2.5. Các vấn đề trọng tâm cần giải quyết

Dựa trên thông tin khảo sát đã thu thập, các vấn đề trọng tâm cần được giải quyết khi xây dựng hệ thống bao gồm:

- Phải xác định rõ cấu trúc và loại thông tin nhập vào hệ thống (họ tên, nhóm máu, địa điểm, thời gian…).
- Cần phân biệt chính xác giữa dữ liệu lưu trữ, dữ liệu hiển thị (tra cứu) và dữ liệu xuất ra báo cáo (thống kê số liệu phân loại theo nhóm máu, khu vực, xuất danh sách đăng ký…).
- Đảm bảo ràng buộc dữ liệu: mỗi người chỉ được đăng ký một lần cho mỗi đợt hiến máu, mỗi sự kiện có giới hạn số lượng người đăng ký… để đảm bảo tính hợp lệ và chính xác của hệ thống.
- Tất cả các quy trình đăng ký, duyệt, cập nhật, thống kê, phản hồi đều phải đảm bảo tính bảo mật, minh bạch và tiện lợi cho cả người dùng lẫn đơn vị tổ chức.
- Xây dựng hệ thống website có giao diện responsive, đảm bảo người dùng dễ dàng tiếp cận trên máy tính, điện thoại thông minh, đáp ứng đúng thực tế và dễ dàng mở rộng tích hợp về sau phù hợp với nhu cầu vận động hiến máu của các tổ chức tại Việt Nam.

3. PHÂN TÍCH HỆ THỐNG

3.1. Xác định yêu cầu hệ thống

Giai đoạn phân tích hệ thống nhằm xác định đầy đủ các thông tin và chức năng xử lý của hệ thống, đảm bảo đáp ứng đúng yêu cầu thực tiễn. Cụ thể:

Xác định các chức năng chính và phụ của hệ thống:

a, Quản lý người dùng

- Đăng ký, đăng nhập, đăng xuất
- Cập nhật thông tin cá nhân, đổi mật khẩu
- Xem lịch sử hiến máu, nhận thông báo

b, Quản lý sự kiện hiến máu

- Tạo, cập nhật, xoá sự kiện
- Quản lý số lượng người đăng ký, trạng thái sự kiện

c, Đăng ký hiến máu

- Đăng ký/hủy đăng ký tham gia sự kiện
- Kiểm tra điều kiện tham gia, check-in tại sự kiện

d, Sàng lọc sức khỏe

- Nhập kết quả kiểm tra sức khỏe, đánh giá điều kiện hiến máu
- Lưu lý do loại (nếu không đủ điều kiện)

e, Lưu trữ lịch sử hiến máu

- Ghi nhận lịch sử hiến máu (thể tích, loại máu, thời gian)
- Tính ngày đủ điều kiện lần hiến tiếp theo

f, Quản lý tin tức & thông báo

- Đăng, cập nhật bài viết/tin tức
- Quản lý trạng thái xuất bản, gửi thông báo

g, Cấu hình hệ thống

- Quản lý, cập nhật các giá trị cấu hình hệ thống

h, Báo cáo & thống kê

- Thống kê số lượng người hiến máu, đăng ký từng sự kiện
- Phân tích nhóm máu, xuất báo cáo

Xác định các nghiệp vụ cần xử lý, đảm bảo:

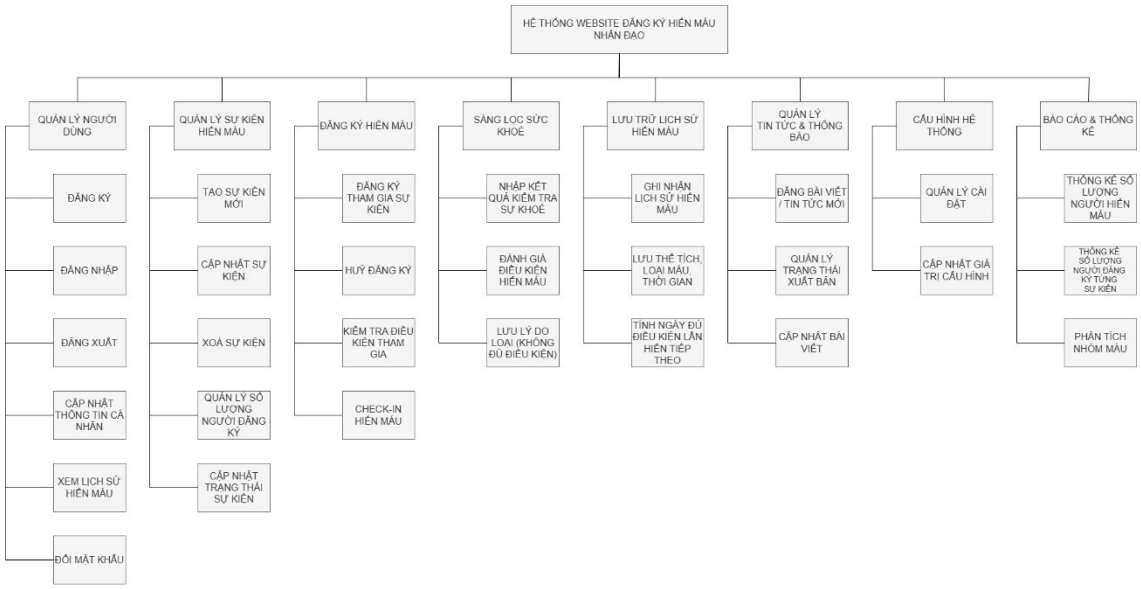
- Đúng quy trình tổ chức hiến máu nhân đạo.
- Tuân thủ quy định y tế, bảo mật thông tin cá nhân.

Yêu cầu về hiệu năng và mở rộng:

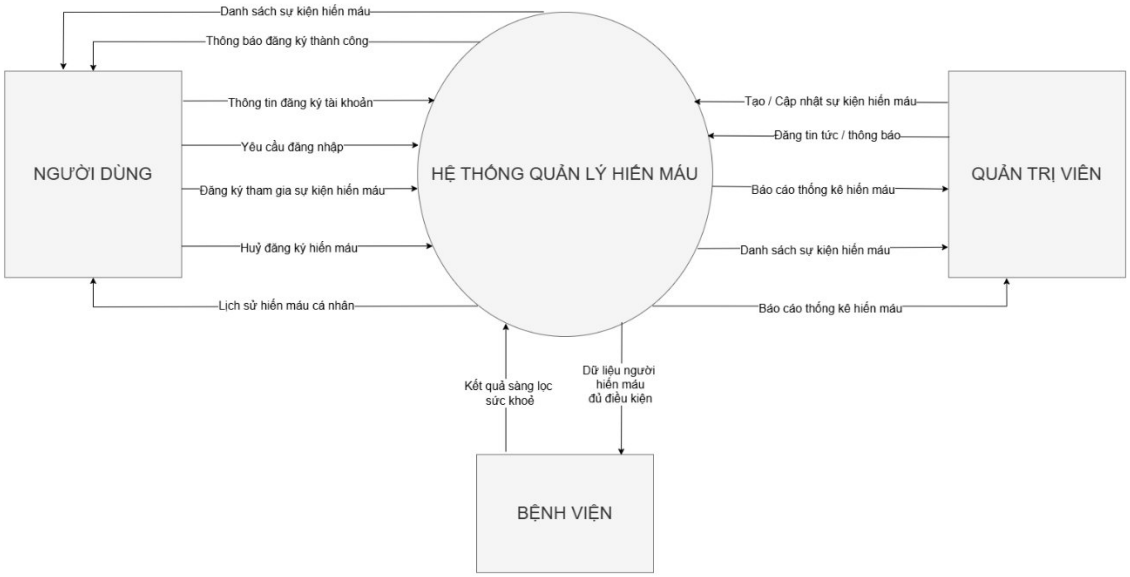
- Hệ thống cần xử lý nhanh, hỗ trợ nhiều người dùng.
- Có khả năng mở rộng vùng địa lý, số lượng sự kiện.
- Giao diện thân thiện, dễ sử dụng trên mọi thiết bị.
- Bảo mật ở nhiều tầng (dữ liệu cá nhân, phân quyền truy cập).
- Độ tin cậy (sao lưu dữ liệu, phòng ngừa sự cố).
- Dễ tích hợp với các dịch vụ khác trong tương lai (email, SMS, bản đồ…).

3.2. Mô hình

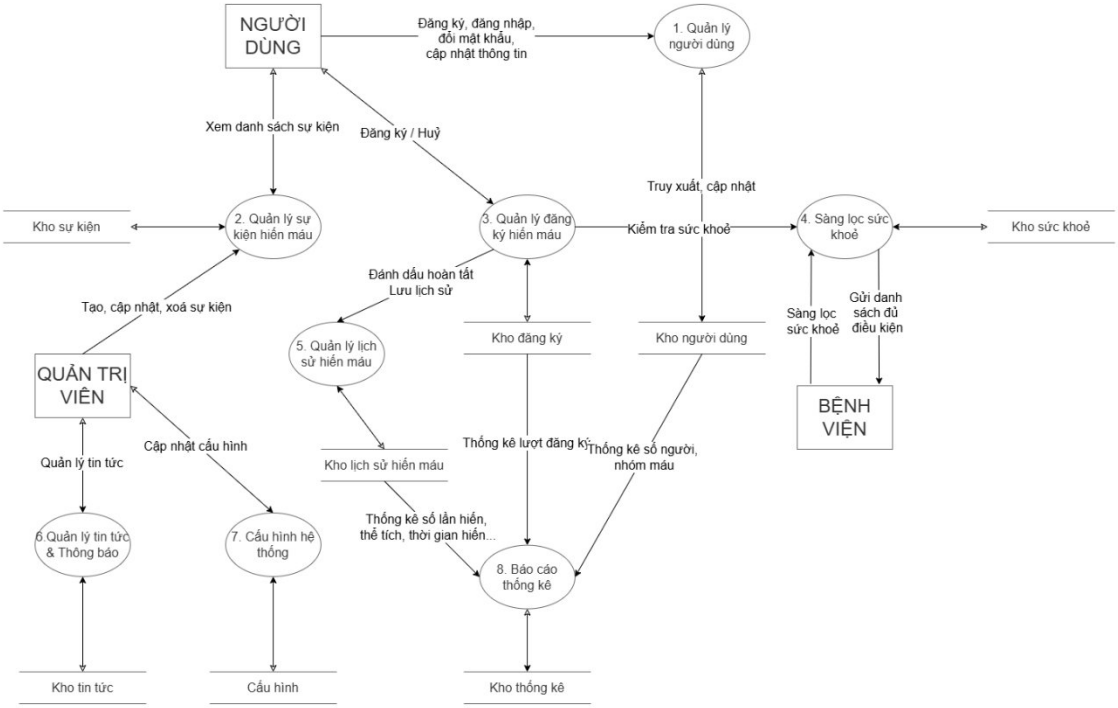
3.2.1. Sơ đồ phân rã chức năng



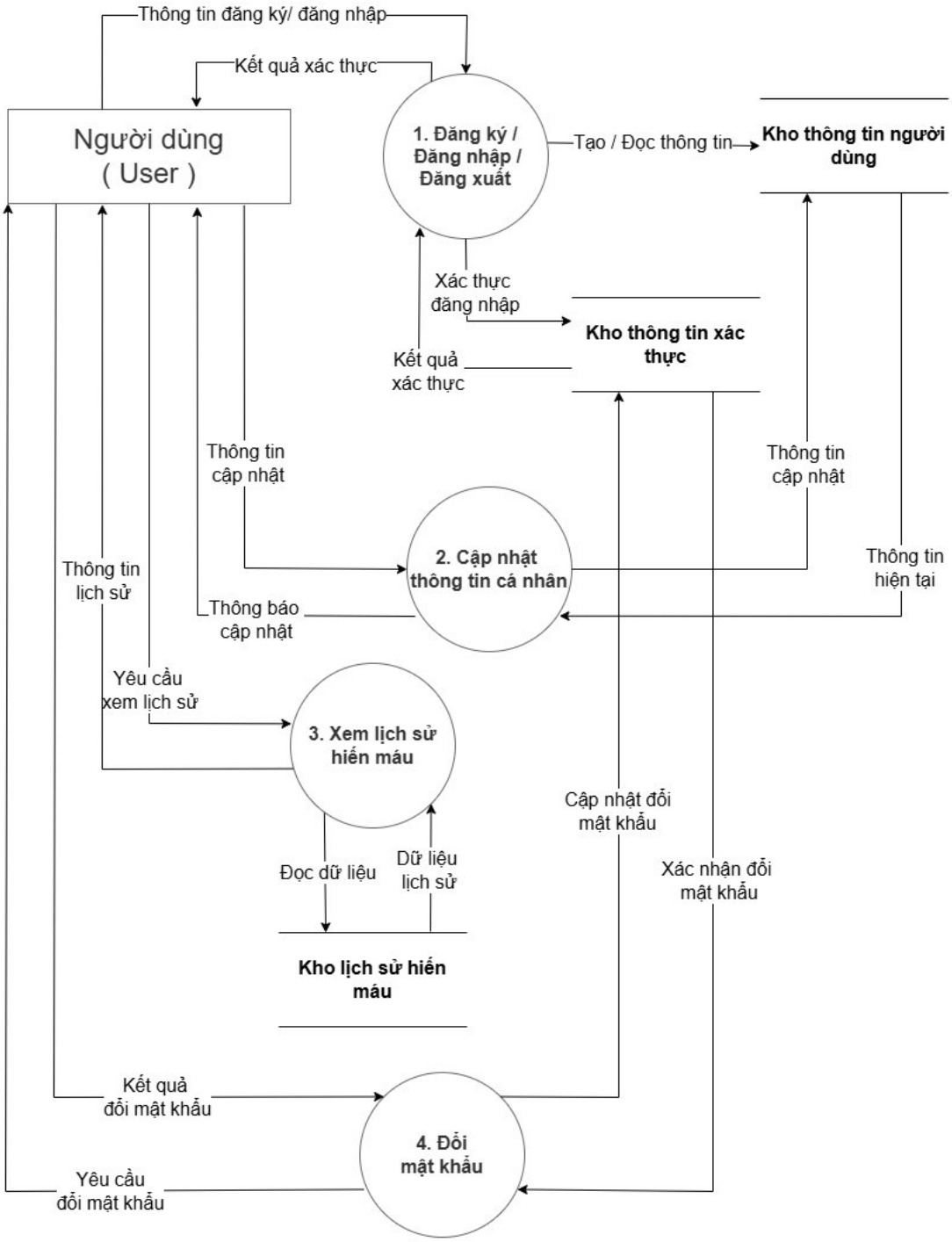
3.2.2. Sơ đồ luồng dữ liệu mức đỉnh



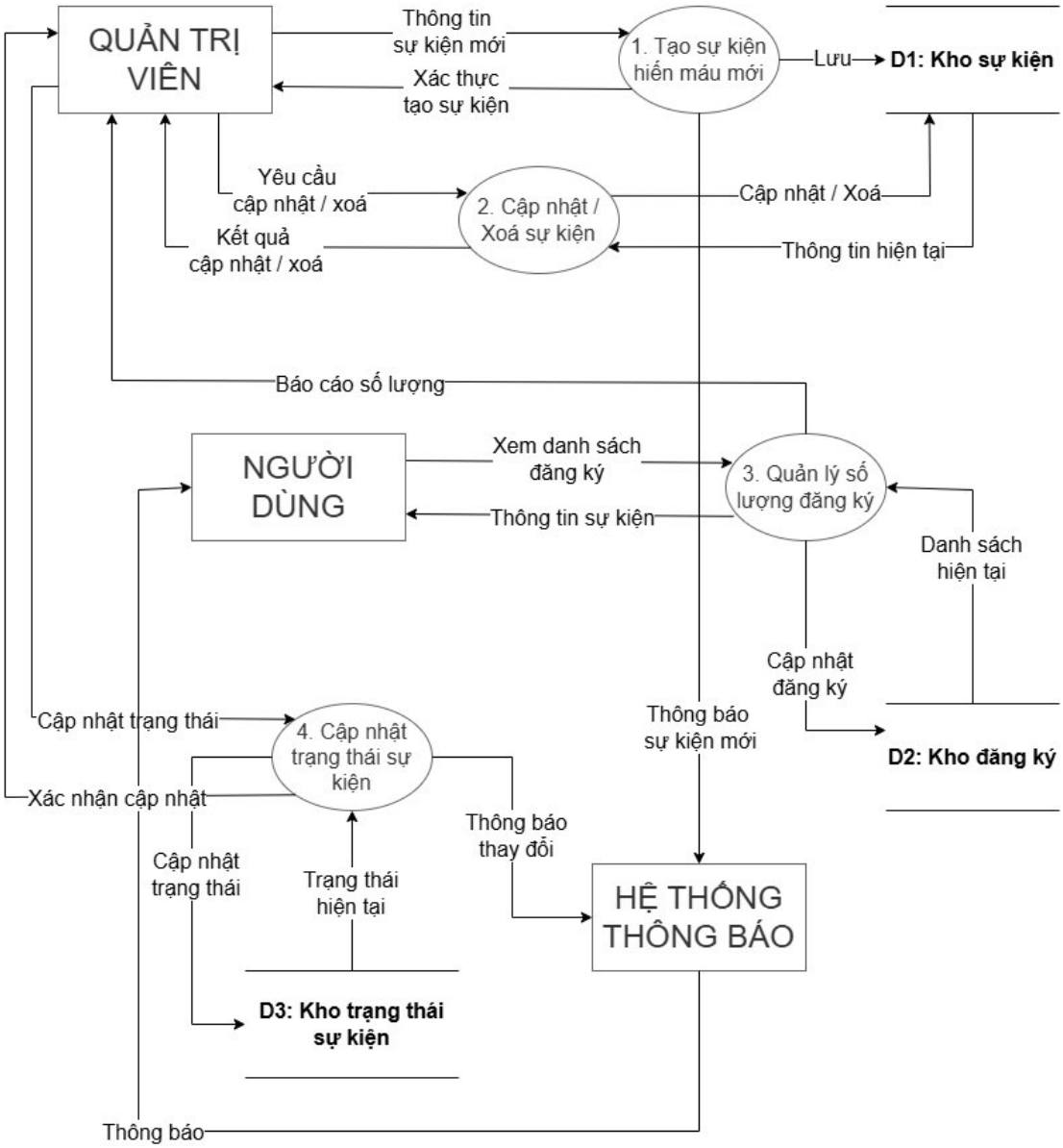
3.2.3. Sơ đồ luồng dữ liệu mức dưới đỉnh



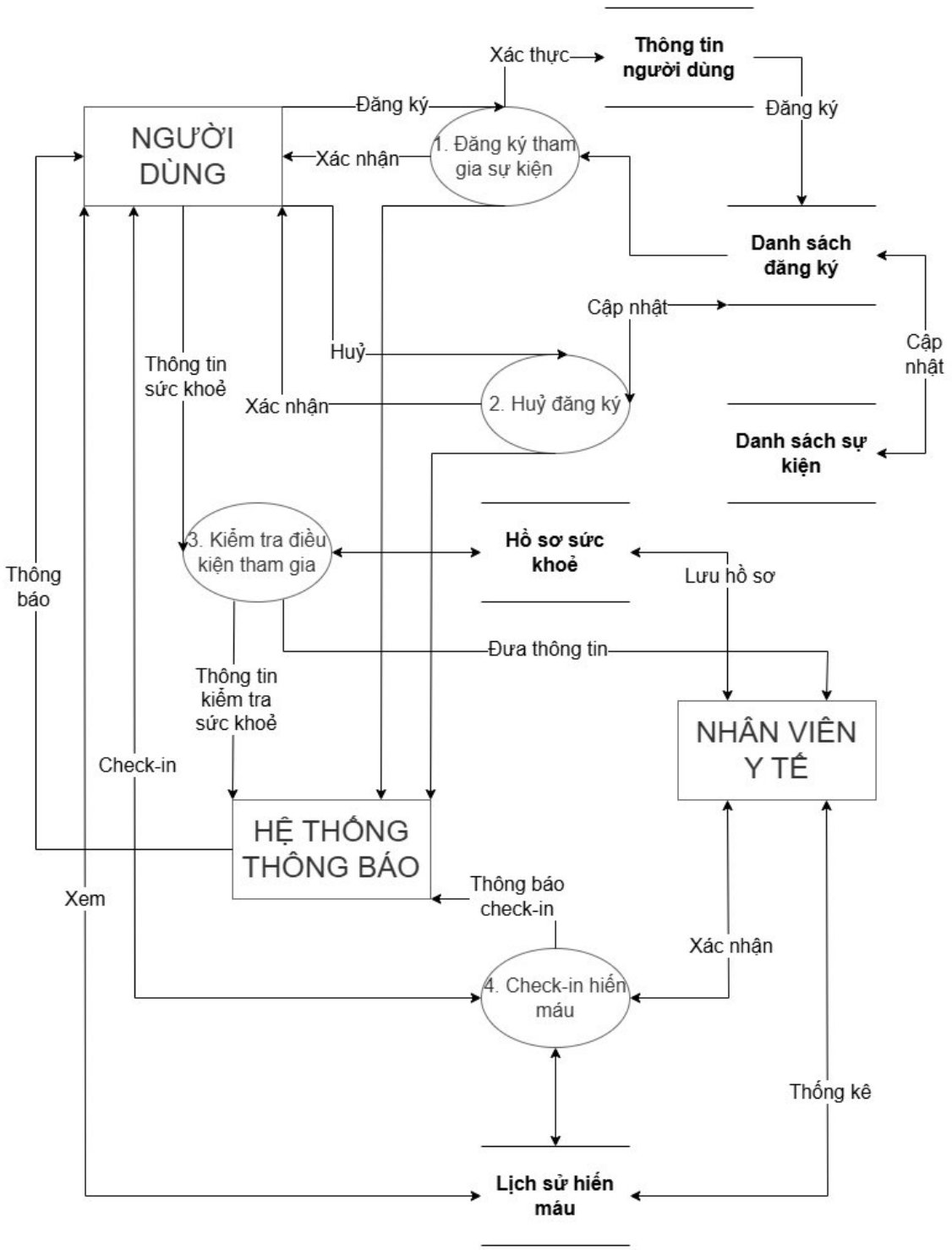
3.2.4. Sơ đồ luồng dữ liệu quản lý người dùng



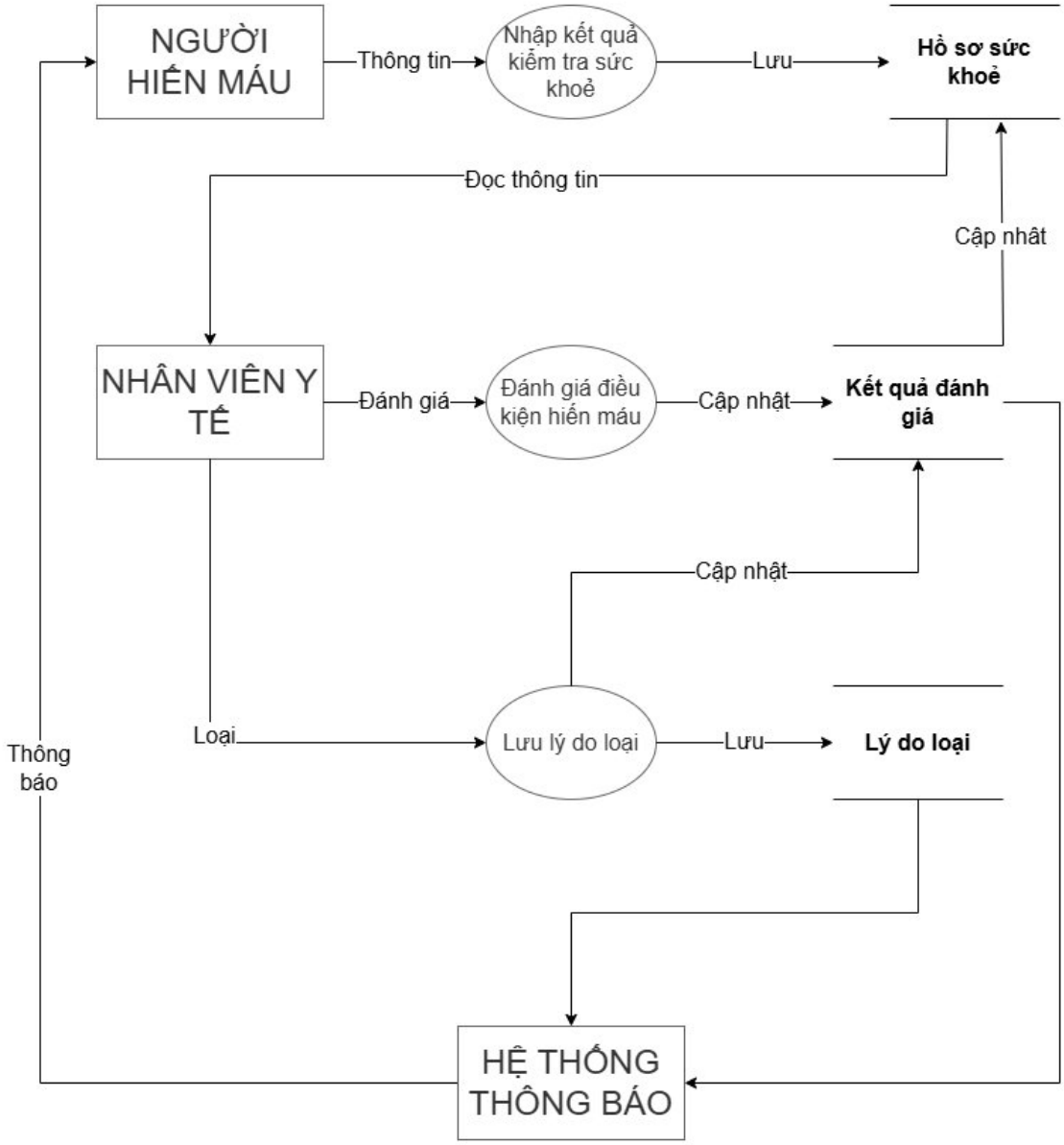
3.2.5. Sơ đồ luồng dữ liệu quản lý sự kiện hiến máu



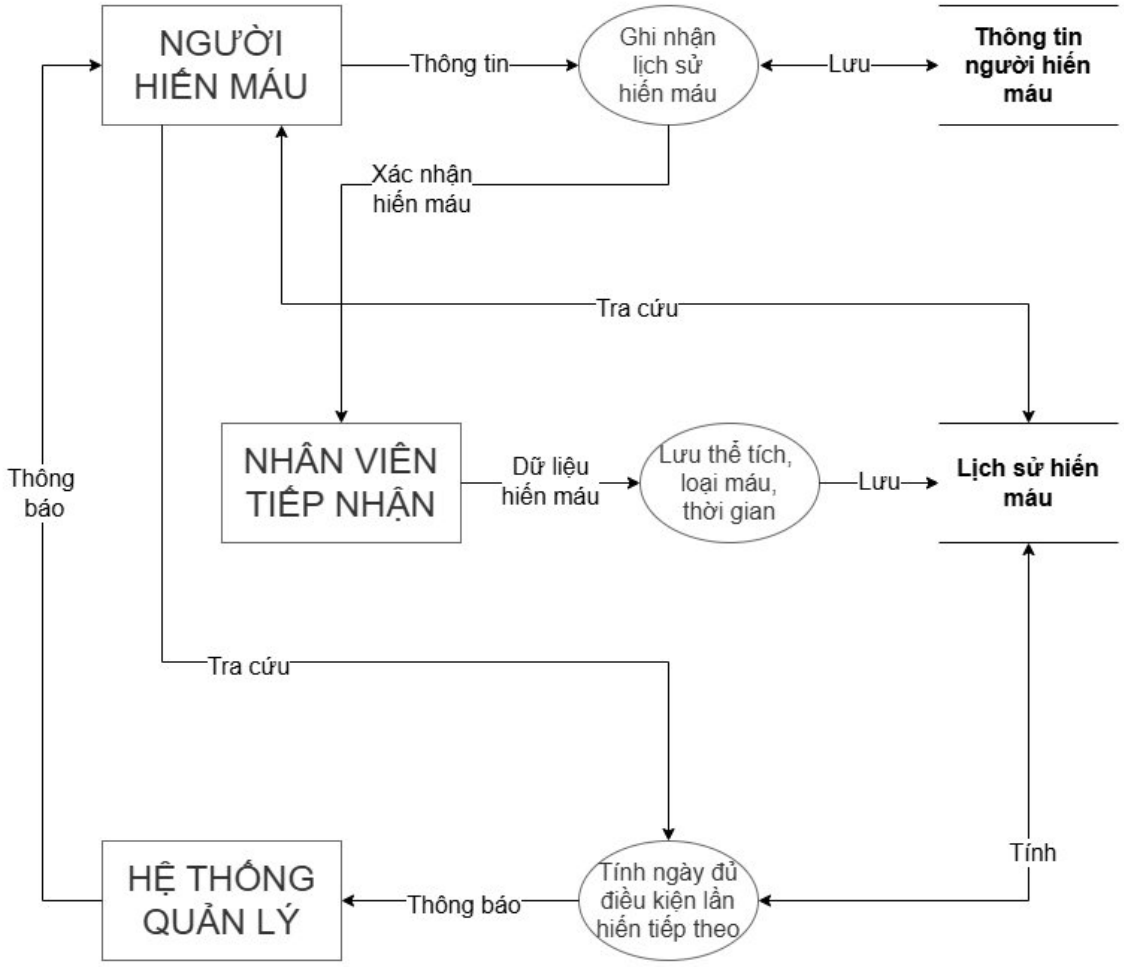
3.2.6. Sơ đồ luồng dữ liệu đăng ký hiến máu



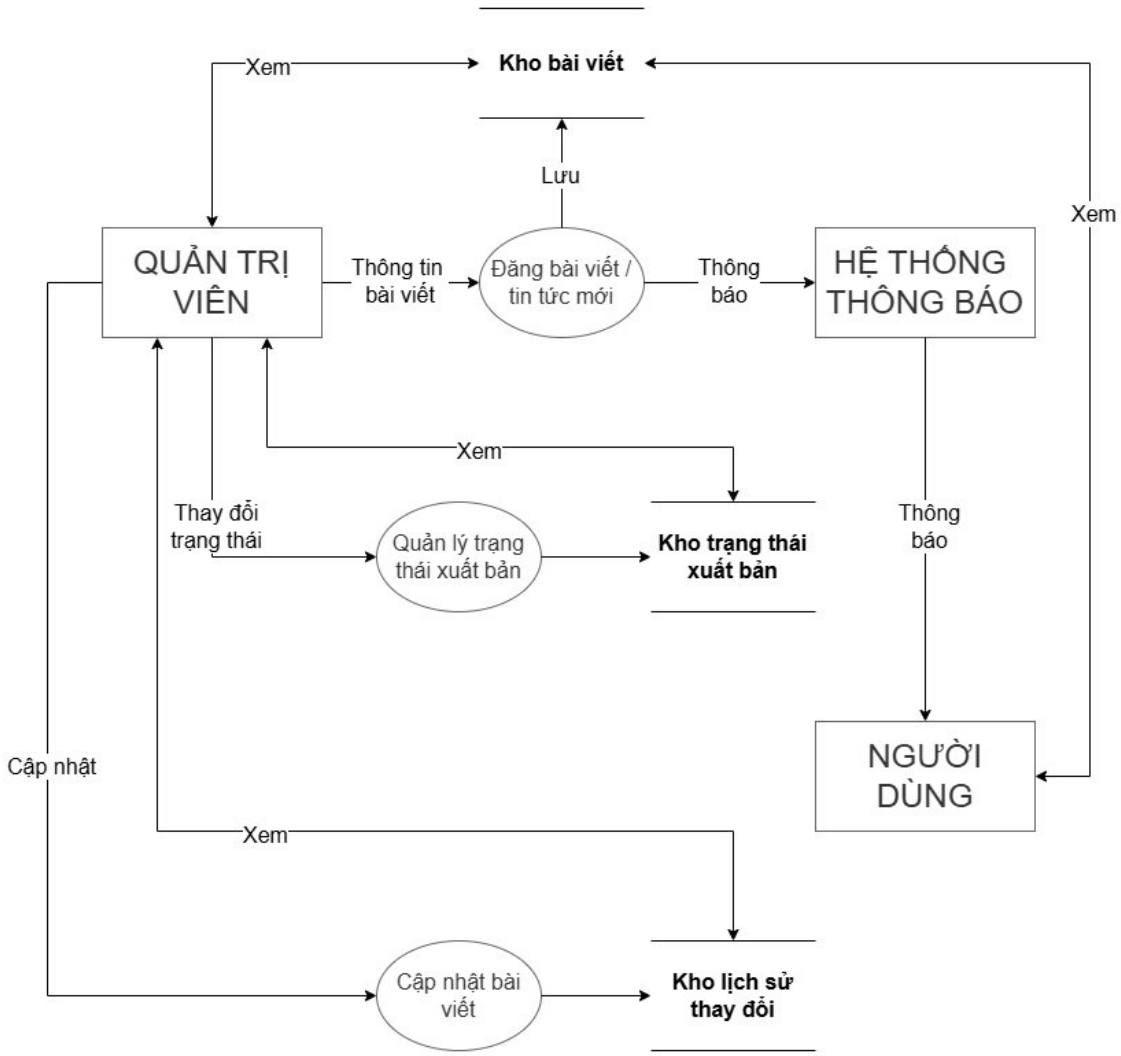
3.2.7. Sơ đồ luồng dữ liệu sàng lọc sức khỏe



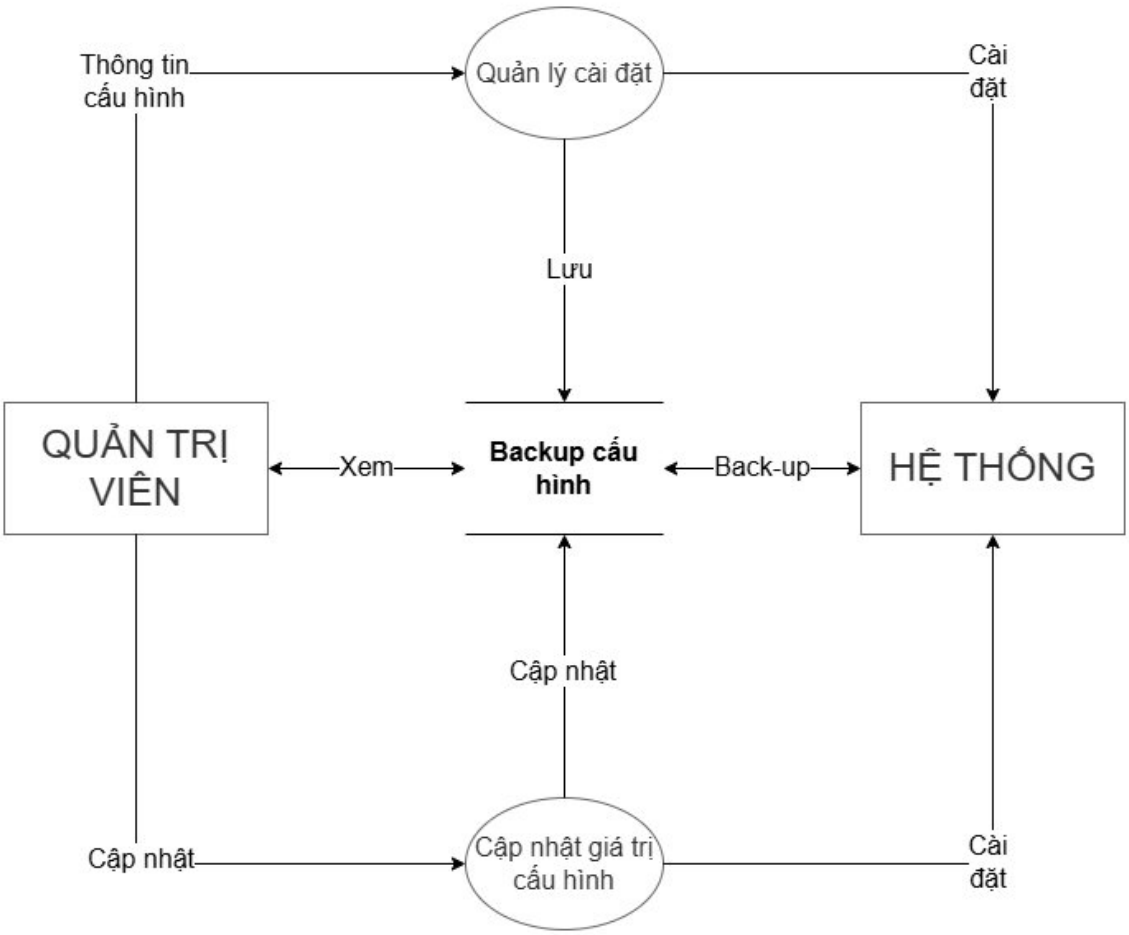
3.2.8. Sơ đồ luồng dữ liệu lưu trữ lịch sử hiến máu



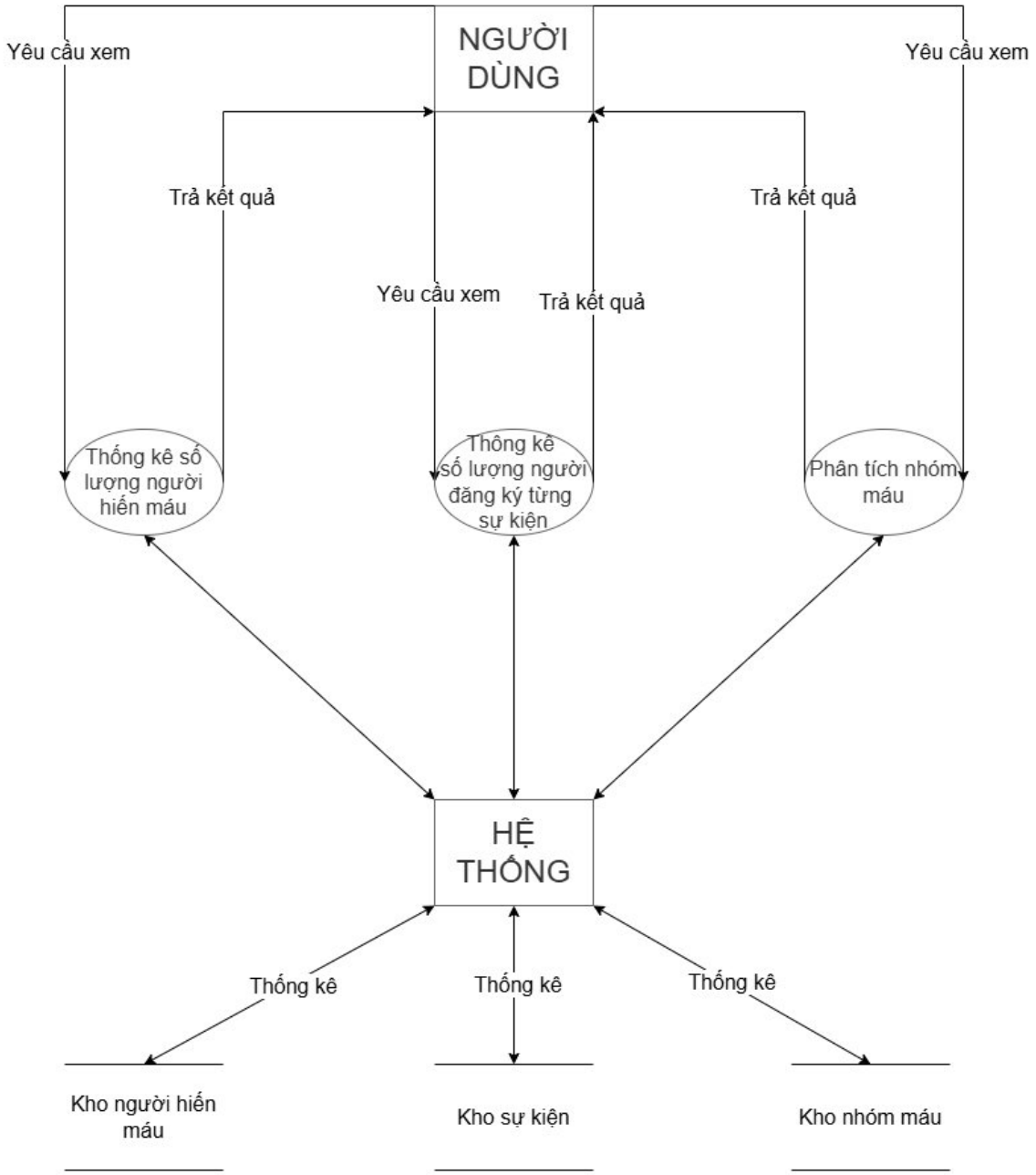
3.2.9. Sơ đồ luồng dữ liệu quản lý tin tức & thông báo



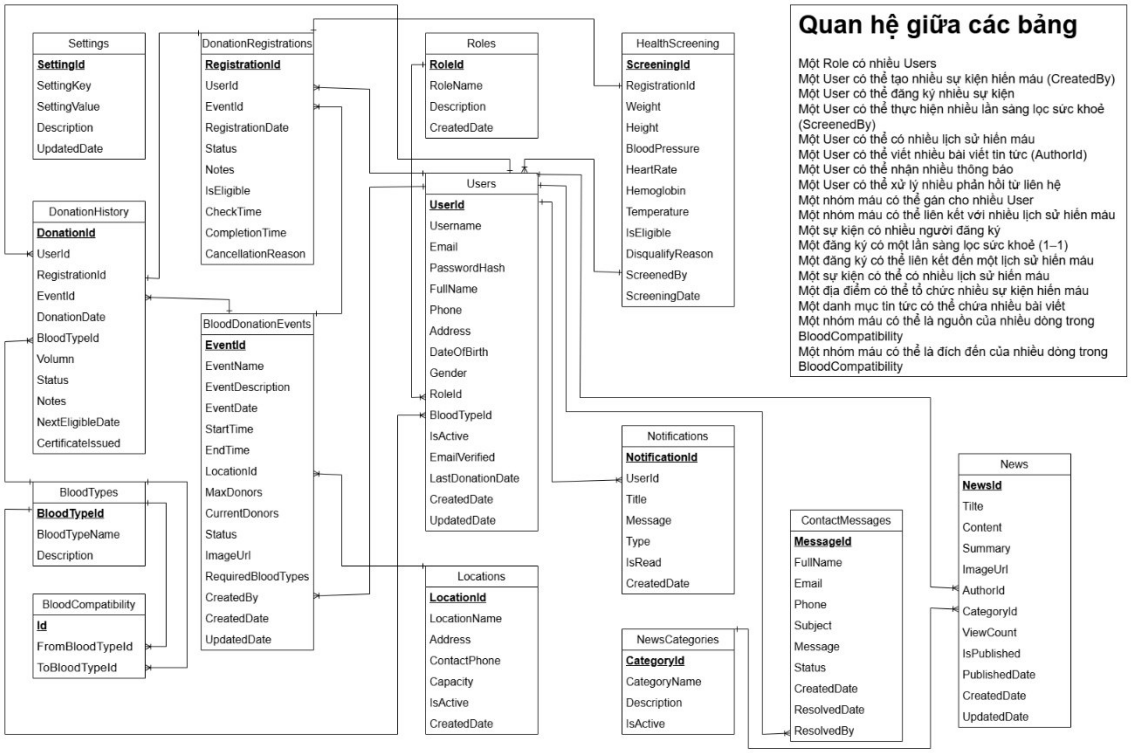
3.2.10. Sơ đồ luồng dữ liệu cấu hình hệ thống



3.2.11. Sơ đồ luồng dữ liệu báo cáo & thống kê



3.2.12. Sơ đồ thực thể quan hệ


Settings
SettingId
SettingKey
SettingValue
Description
UpdatedDate
DonationRegistrations
RegistrationId
UserId
EventId
RegistrationDate
Status
Notes
IsEligible
CheckTime
CompletionTime
CancellationReason
Roles
RoleId
RoleName
Description
CreatedDate
HealthScreening
ScreeningId
RegistrationId
Weight
Height
BloodPressure
HeartRate
Hemoglobin
Temperature
IsEligible
DisqualifyReason
ScreenedBy
ScreeningDate
Users
UserId
Username
Email
PasswordHash
FullName
Phone
Address
DateOfBirth
Gender
RoleId
BloodTypeId
IsActive
EmailVerified
LastDonationDate
CreatedDate
UpdatedDate
DonationHistory
DonationId
UserId
RegistrationId
EventId
DonationDate
BloodTypeId
Volumn
Status
Notes
NextEligibleDate
CertificateIssued
BloodDonationEvents
EventId
EventName
EventDescription
EventDate
StartTime
EndTime
LocationId
MaxDonors
CurrentDonors
Status
ImageUrl
RequiredBloodTypes
CreatedBy
CreatedDate
UpdatedDate
BloodTypes
BloodTypeId
BloodTypeName
Description
BloodCompatibility
Id
FromBloodTypeId
ToBloodTypeId
Locations
LocationId
LocationName
Address
ContactPhone
Capacity
IsActive
CreatedDate
Notifications
NotificationId
UserId
Title
Message
Type
IsRead
CreatedDate
NewsCategories
CategoryId
CategoryName
Description
IsActive
ContactMessages
MessageId
FullName
Email
Phone
Subject
Message
Status
CreatedDate
ResolvedDate
ResolvedBy
News
NewsId
Tilte
Content
Summary
ImageUrl
AuthorId
CategoryId
ViewCount
IsPublished
PublishedDate
CreatedDate
UpdatedDate
Quan hệ giữa các bảng
Một Role có nhiều Users
Một User có thể tạo nhiều sự kiện hiến máu (CreatedBy)
Một User có thể đăng ký nhiều sự kiện
Một User có thể thực hiện nhiều lần sàng lọc sức khoẻ (ScreenedBy)
Một User có thể có nhiều lịch sử hiến máu
Một User có thể viết nhiều bài viết tin tức (AuthorId)
Một User có thể nhận nhiều thông báo
Một User có thể xử lý nhiều phản hồi từ liên hệ
Một nhóm máu có thể gán cho nhiều User
Một nhóm máu có thể liên kết với nhiều lịch sử hiến máu
Một sự kiện có nhiều người đăng ký
Một đăng ký có một lần sàng lọc sức khoẻ (1–1)
Một đăng ký có thể liên kết đến một lịch sử hiến máu
Một sự kiện có thể có nhiều lịch sử hiến máu
Một địa điểm có thể tổ chức nhiều sự kiện hiến máu
Một danh mục tin tức có thể chứa nhiều bài viết
Một nhóm máu có thể là nguồn của nhiều dòng trong BloodCompatibility
Một nhóm máu có thể là đích đến của nhiều dòng trong BloodCompatibility

3.3. Phân tích cơ sở dữ liệu

3.3.1. Danh sách các bảng dữ liệu chính

- Roles: Quản lý vai trò của người dùng trong hệ thống
- Users: Thông tin người dùng
- BloodTypes: Danh sách các nhóm máu
- Locations: Địa điểm tổ chức hiến máu
- BloodDonationEvents: Sự kiện hiến máu được tổ chức tại địa điểm
- DonationRegistrations: Thông tin đăng ký tham gia hiến máu
- HealthScreening: Kết quả kiểm tra sức khoẻ trước khi hiến máu
- DonationHistory: Lịch sử hiến máu của từng người
- NewsCategories: Danh mục tin tức liên quan đến hiến máu
- News: Tin tức, bài viết về hoạt động hiến máu
- Notifications: Thông báo gửi đến người dùng
- Settings: Các cấu hình và thiết lập hệ thống
- ContactMessages: Tin nhắn liên hệ người dùng gửi đến hệ thống

3.3.2. Chi tiết

a. Bảng Roles

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
RoleId	int (PK, tăng tự động)	Khóa chính, định danh duy nhất cho vai trò
RoleName	nvarchar(50)	Tên vai trò (Admin, User, Hospital, Doctor, Staff), dùng cho phân quyền
Description	nvarchar(200)	Mô tả chi tiết về vai trò
CreatedDate	datetime	Ngày tạo vai trò

b. Bảng Users

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
UserId	int (PK, tăng tự động)	Khóa chính, định danh duy nhất cho người dùng
Username	nvarchar(50)	Tên đăng nhập, duy nhất
Email	nvarchar(100)	Email người dùng, duy nhất
PasswordHash	nvarchar(255)	Mã hóa mật khẩu người dùng
FullName	nvarchar(100)	Họ tên đầy đủ của người dùng
Phone	nvarchar(15)	Số điện thoại người dùng
Address	nvarchar(255)	Địa chỉ người dùng
DateOfBirth	date	Ngày sinh
Gender	nvarchar(10)	Giới tính (Nam, Nữ, Khác)
BloodTypeId	int	Nhóm máu, liên kết đến bảng BloodTypes
RoleId	int	Vai trò, liên kết đến bảng Roles
IsActive	bit	Trạng thái hoạt động của tài khoản (1: hoạt động, 0: khóa)
EmailVerified	bit	Đã xác thực email hay chưa
LastDonationDate	date	Ngày hiến máu gần nhất
CreatedDate	datetime	Ngày tạo tài khoản
UpdatedDate	datetime	Ngày cập nhật tài khoản gần nhất

c. Bảng BloodTypes

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
BloodTypeId	int (PK, tăng tự động)	Khóa chính, định danh nhóm máu
BloodTypeName	nvarchar(5)	Tên nhóm máu (A, B, AB, O, v.v.)
Description	nvarchar(100)	Mô tả về nhóm máu

d. Bảng BloodCompatibility

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
Id	int (PK, tăng tự động)	Khóa chính, định danh duy nhất cho bản ghi tương thích nhóm máu
FromBloodTypeId	int	Nhóm máu cho (từ), liên kết đến BloodTypes
ToBloodTypeId	int	Nhóm máu nhận (đến), liên kết đến BloodTypes

e. Bảng Locations

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
LocationId	int (PK, tăng tự động)	Khóa chính, định danh địa điểm
LocationName	nvarchar(200)	Tên địa điểm tổ chức hiến máu
Address	nvarchar(500)	Địa chỉ chi tiết địa điểm
ContactPhone	nvarchar(15)	Số điện thoại liên hệ
Capacity	int	Sức chứa tối đa của địa điểm
IsActive	bit	Địa điểm còn hoạt động hay không
CreatedDate	datetime	Ngày tạo địa điểm

f. Bảng BloodDonationEvents

Tên thuộc tính	**Kiểu dữ liệu**	**Vai trò/Ý nghĩa**
EventId	int (PK, tăng tự động)	Khóa chính, định danh sự kiện hiến máu
EventName	nvarchar(200)	Tên sự kiện hiến máu
EventDescription	nvarchar(max)	Mô tả chi tiết về sự kiện
EventDate	date	Ngày diễn ra sự kiện
StartTime	time	Thời gian bắt đầu
EndTime	time	Thời gian kết thúc
LocationId	int	Địa điểm tổ chức, liên kết đến Locations
MaxDonors	int	Số lượng người hiến tối đa
CurrentDonors	int	Số lượng người đã đăng ký/đã hiến
Status	nvarchar(20)	Trạng thái sự kiện (Draft, Active, Completed, ...)
ImageUrl	nvarchar(255)	Đường dẫn ảnh minh họa sự kiện
RequiredBloodTypes	nvarchar(100)	Nhóm máu cần thiết cho sự kiện
CreatedBy	int	Người tạo sự kiện, liên kết đến Users
CreatedDate	datetime	Ngày tạo sự kiện
UpdatedDate	datetime	Ngày cập nhật sự kiện

g. Bảng DonationRegistrations

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
RegistrationId	int (PK, tăng tự động)	Khóa chính, định danh lượt đăng ký
UserId	int	Người đăng ký, liên kết đến Users
EventId	int	Sự kiện đăng ký, liên kết đến BloodDonationEvents
RegistrationDate	datetime	Ngày giờ đăng ký
Status	nvarchar(20)	Trạng thái đăng ký (Registered, Confirmed, CheckedIn, ...)
Notes	nvarchar(500)	Ghi chú thêm
IsEligible	bit	Đủ điều kiện hiến máu hay không
CheckInTime	datetime	Thời gian check-in tại sự kiện
CompletionTime	datetime	Thời gian hoàn thành hiến máu
CancellationReason	nvarchar(200)	Lý do hủy đăng ký (nếu có)

h. Bảng HealthScreening

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
ScreeningId	int (PK, tăng tự động)	Khóa chính, định danh lượt sàng lọc
RegistrationId	int	Liên kết đến lượt đăng ký (DonationRegistrations)
Weight	decimal(5,2)	Cân nặng người hiến
Height	decimal(5,2)	Chiều cao người hiến
BloodPressure	nvarchar(20)	Huyết áp
HeartRate	int	Nhịp tim
Temperature	decimal(4,2)	Nhiệt độ cơ thể
Hemoglobin	decimal(4,2)	Nồng độ Hemoglobin
IsEligible	bit	Đủ điều kiện hiến máu hay không
DisqualifyReason	nvarchar(500)	Lý do không đủ điều kiện (nếu có)
ScreenedBy	int	Người thực hiện sàng lọc, liên kết đến Users
ScreeningDate	datetime	Ngày thực hiện sàng lọc

i. Bảng DonationHistory

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
DonationId	int (PK, tăng tự động)	Khóa chính, định danh lịch sử hiến máu
UserId	int	Người hiến máu, liên kết đến Users
EventId	int	Sự kiện hiến máu, liên kết đến BloodDonationEvents
RegistrationId	int	Liên kết đến lượt đăng ký (có thể null nếu không đăng ký trước)
DonationDate	datetime	Ngày giờ hiến máu
BloodTypeId	int	Nhóm máu của người hiến, liên kết đến BloodTypes
Volume	int	Lượng máu hiến (ml)
Status	nvarchar(20)	Trạng thái hiến máu (Started, Completed, Failed, ...)
Notes	nvarchar(500)	Ghi chú thêm
NextEligibleDate	date	Ngày đủ điều kiện hiến máu tiếp theo
CertificateIssued	bit	Đã cấp giấy chứng nhận hiến máu hay chưa

j. Bảng NewsCategories

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
CategoryId	int (PK, tăng tự động)	Khóa chính, định danh danh mục tin tức
CategoryName	nvarchar(100)	Tên danh mục
Description	nvarchar(200)	Mô tả về danh mục
IsActive	bit	Danh mục còn hoạt động hay không

k. Bảng News

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
NewsId	int (PK, tăng tự động)	Khóa chính, định danh tin tức
Title	nvarchar(200)	Tiêu đề tin tức
Content	nvarchar(max)	Nội dung chi tiết tin tức
Summary	nvarchar(500)	Tóm tắt nội dung
ImageUrl	nvarchar(255)	Đường dẫn ảnh minh họa
CategoryId	int	Danh mục tin tức, liên kết đến NewsCategories
AuthorId	int	Người đăng tin, liên kết đến Users
ViewCount	int	Số lượt xem
IsPublished	bit	Đã xuất bản hay chưa
PublishedDate	datetime	Ngày xuất bản
CreatedDate	datetime	Ngày tạo tin
UpdatedDate	datetime	Ngày cập nhật tin

l. Bảng Notifications

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
NotificationId	int (PK, tăng tự động)	Khóa chính, định danh thông báo
UserId	int	Người nhận thông báo, liên kết đến Users
Title	nvarchar(200)	Tiêu đề thông báo
Message	nvarchar(500)	Nội dung thông báo
Type	nvarchar(50)	Loại thông báo (Registration, Reminder, Info, ...)
IsRead	bit	Đã đọc hay chưa
CreatedDate	datetime	Ngày tạo thông báo

m. Bảng Settings

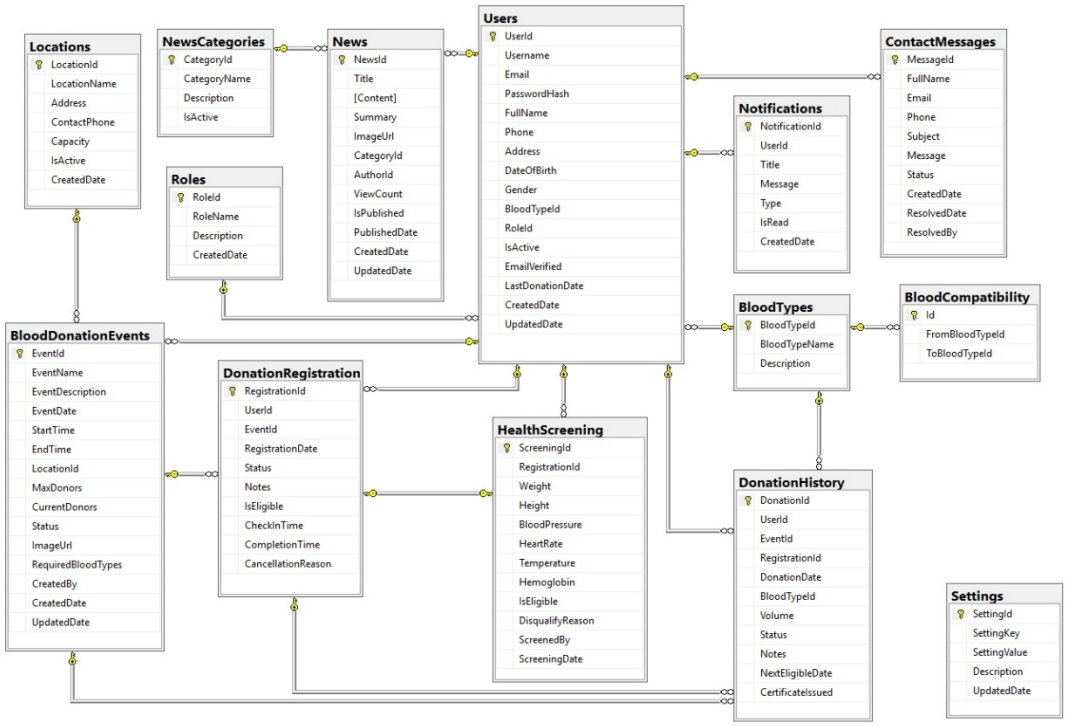
Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
SettingId	int (PK, tăng tự động)	Khóa chính, định danh thiết lập
SettingKey	nvarchar(50)	Khóa thiết lập, duy nhất
SettingValue	nvarchar(255)	Giá trị thiết lập
Description	nvarchar(200)	Mô tả về thiết lập
UpdatedDate	datetime	Ngày cập nhật thiết lập

n. Bảng ContactMessages

Tên thuộc tính	Kiểu dữ liệu	Vai trò/Ý nghĩa
MessageId	int (PK, tăng tự động)	Khóa chính, định danh tin nhắn liên hệ
FullName	nvarchar(100)	Họ tên người gửi
Email	nvarchar(100)	Email người gửi
Phone	nvarchar(15)	Số điện thoại người gửi
Subject	nvarchar(200)	Chủ đề tin nhắn
Message	nvarchar(1000)	Nội dung tin nhắn
Status	nvarchar(20)	Trạng thái xử lý tin nhắn (New, Read, Resolved, ...)
CreatedDate	datetime	Ngày gửi tin nhắn
ResolvedDate	datetime	Ngày xử lý xong tin nhắn
ResolvedBy	int	Người xử lý, liên kết đến Users

3.3.3. Quan hệ giữa các bảng

Bằng chữ	Bằng ký hiệu
Một Role có nhiều Users	Roles 1–N Users
Một User có thể tạo nhiều sự kiện hiến máu	BloodTypes 1–N Users
Một User có thể đăng ký nhiều sự kiện	Locations 1–N BloodDonationEvents
Một User có thể thực hiện nhiều lần sàng lọc sức khoẻ (ScreenedBy)	Users 1–N BloodDonationEvents
Một User có thể có nhiều lịch sử hiến máu	Users 1–N DonationRegistrations
Một User có thể viết nhiều bài viết tin tức (AuthorId)	BloodDonationEvents 1–N DonationRegistrations
Một User có thể nhận nhiều thông báo	BloodTypes 1–N DonationHistory
Một User có thể xử lý nhiều phản hồi từ liên hệ	DonationRegistrations 1–1 HealthScreening
Một nhóm máu có thể gán cho nhiều User	Users 1–N HealthScreening
Một nhóm máu liên kết với nhiều lịch sử hiến máu	Users 1–N DonationHistory
Một sự kiện có nhiều người đăng ký	BloodDonationEvents 1–N DonationHistory
Một đăng ký có một lần sàng lọc sức khoẻ (1 - 1)	DonationRegistrations 1–N DonationHistory
Một đăng ký liên kết đến một lịch sử hiến máu	NewsCategories 1–N News
Một sự kiện có thể có nhiều lịch sử hiến máu	Users 1–N News
Một địa điểm có thể tổ chức nhiều sự kiện hiến máu	Users 1–N Notifications
Một danh mục tin tức có thể chứa nhiều bài viết	Users 1–N ContactMessages
Một nhóm máu có thể là nguồn của nhiều dòng trong BloodCompatibility	BloodTypes 1–N BloodCompatibility
Một nhóm máu có thể là đích đến của nhiều dòng trong BloodCompatibility	BloodTypes 1–N BloodCompatibility



3.3.4. Ràng buộc dữ liệu

- BloodTypes: Chỉ chấp nhận các giá trị nhóm máu chuẩn như A, B, AB, O và các biến thể có Rh như A+, A−, O+, O−,…
- Giới tính (Gender): Chỉ cho phép ba giá trị: "Male", "Female", hoặc "Other". Không để trống, không viết tự do.
- Số điện thoại: Phải từ 9 đến 15 ký tự, chỉ chứa chữ số. Không chấp nhận ký tự đặc biệt, chữ cái hay khoảng trắng.
- Thời gian sự kiện (StartTime, EndTime): Thời gian bắt đầu phải nhỏ hơn thời gian kết thúc. Không được tạo sự kiện bắt đầu sau khi đã kết thúc.
- Khối lượng máu hiến (Volume): Chỉ chấp nhận trong khoảng 250–500 ml. Dưới mức đó thì không đủ chuẩn, trên thì nguy hiểm.
- Trường boolean (IsActive, IsPublished, IsRead...): Chỉ nhận giá trị 0 hoặc 1. Không được để null hoặc bất kỳ thứ gì khác.
- Ngày tháng (LastDonationDate, DonationDate...): Không được lớn hơn ngày hiện tại. Không thể hiến máu trong tương lai.
- Tên người dùng, email, nhóm máu...: Phải là duy nhất, không được trùng lặp trong hệ thống.

- Ràng buộc dữ liệu liên kết: Không được xóa người dùng nếu họ đã tham gia đăng ký hoặc có lịch sử hiến máu.
- DonationRegistrations.Status: Chỉ nhận các giá trị: 'Registered', 'Confirmed', 'CheckedIn', 'Eligible', 'Ineligible', 'Donating', 'Completed', 'Cancelled', 'NoShow', 'Failed'. Trong đó:
  - Registered: Đã đăng ký nhưng chưa xác nhận.
  - Confirmed: Đã xác nhận tham gia.
  - CheckedIn: Đã check-in tại điểm hiến máu.
  - Screening: Đang chờ hoặc đang sàng lọc sức khỏe.
  - Eligible: Đủ điều kiện hiến máu (sau khi sàng lọc).
  - Ineligible: Không đủ điều kiện hiến máu (sau khi sàng lọc).
  - Donating: Đang hiến máu.
  - Completed: Đã hoàn thành hiến máu.
  - Cancelled: Đã hủy đăng ký.
  - Failed: Thất bại (ví dụ: không lấy được máu).
  - NoShow: Không đến.
  - Deferred: Hoãn lại.
- HealthScreening.DisqualifyReason: Nếu IsEligible = 0 thì trường này bắt buộc có, giá trị thuộc: 'LowHemoglobin', 'HighBloodPressure', 'LowBloodPressure', 'Fever', 'LowWeight', 'RecentDonation', 'MedicalHistory', 'CurrentMedication', 'RecentVaccination', 'Pregnancy', 'Breastfeeding', 'RecentSurgery', 'InfectionRisk', 'Other'. Trong đó:
  - LowHemoglobin: Hb thấp
  - HighBloodPressure :Huyết áp cao
  - LowBloodPressure :Huyết áp thấp
  - Fever :Sốt
  - LowWeight :Cân nặng không đủ
  - RecentDonation :Mới hiến gần đây
  - MedicalHistory :Tiền sử bệnh lý
  - CurrentMedication :Đang dùng thuốc
  - RecentVaccination :Mới tiêm vaccine

- Pregnancy :Đang mang thai
- Breastfeeding :Đang cho con bú
- RecentSurgery :Mới phẫu thuật
- InfectionRisk :Nguy cơ nhiễm trùng
- Other :Lý do khác

- BloodDonationEvents.Status: 'Draft', 'Published', 'Active', 'Completed', 'Cancelled', 'Postponed', 'Full', 'Closed'. Trong đó:
  - Draft :Bản nháp
  - Published :Đã công bố
  - Active :Đang diễn ra
  - Completed :Hoàn thành
  - Cancelled :Đã hủy
  - Postponed :Hoãn lại
  - Full :Đã đầy
  - Closed :Đóng đăng ký
- DonationHistory.Status: 'Started', 'InProgress', 'Completed', 'Stopped', 'Failed'. Trong đó:
  - Started: Bắt đầu
  - InProgress: Đang thực hiện
  - Completed: Hoàn thành
  - Stopped: Dừng giữa chừng
  - Failed: Thất bại
- Notifications.Type: 'Registration', 'Confirmation', 'Reminder', 'Cancellation', 'Completion', 'Result', 'Event', 'System', 'Medical', 'Warning', 'Info'. Trong đó:
  - Registration: Đăng ký
  - Confirmation: Xác nhận
  - Reminder: Nhắc nhở
  - Cancellation: Hủy bỏ
  - Completion: Hoàn thành
  - Result: Kết quả
  - Event: Sự kiện

- System: Hệ thống
- Medical: Y tế
- Warning: Cảnh báo
- Info: Thông tin

- ContactMessages.Status: 'New', 'Read', 'InProgress', 'Resolved', 'Closed'.

Trong đó:

- New: Mới
- Read: Đã đọc
- InProgress: Đang xử lý
- Resolved: Đã giải quyết
- Closed: Đã đóng

- CheckInTime: Chỉ nhập được nếu ngày hiện tại đúng với ngày sự kiện.

Không cho phép trước hoặc sau thời gian hợp lệ.

- NextEligibleDate: Phải >= DonationDate và phải thỏa mãn thời gian nghỉ giữa các lần hiến máu theo quy định.
- Volume: Nếu lưu mặc định là 350ml, thì cần khớp giữa ghi nhận và thực tế, không để trống.
- IsEligible (trong nhiều bảng): Phải đồng nhất giữa HealthScreening và DonationRegistrations.

4. THIẾT KẾ

4.1. Mục tiêu thiết kế:

Giai đoạn thiết kế của dự án *Website đăng ký hiến máu* nhằm mục đích xây dựng một hệ thống ứng dụng web hiệu quả, dễ bảo trì và có khả năng mở rộng để hỗ trợ các hoạt động liên quan đến hiến máu. Các mục tiêu thiết kế chính bao gồm:

- Tính mô-đun: Đảm bảo sự phân tách rõ ràng giữa các thành phần để hỗ trợ phát triển và bảo trì độc lập.
- Khả năng mở rộng: Thiết kế hệ thống có khả năng xử lý số lượng người dùng và dữ liệu tăng trưởng mà không ảnh hưởng đến hiệu suất.
- Tính bảo mật: Tích hợp các cơ chế xác thực và phân quyền mạnh mẽ để bảo vệ dữ liệu nhạy cảm.
- Tính thân thiện với người dùng: Cung cấp giao diện người dùng trực quan, dễ sử dụng và đáp ứng nhanh chóng.
- Tính tái sử dụng: Sử dụng các mẫu thiết kế và cấu trúc để tối ưu hóa việc tái sử dụng mã nguồn.

4.2. Mô tả kiến trúc hệ thống

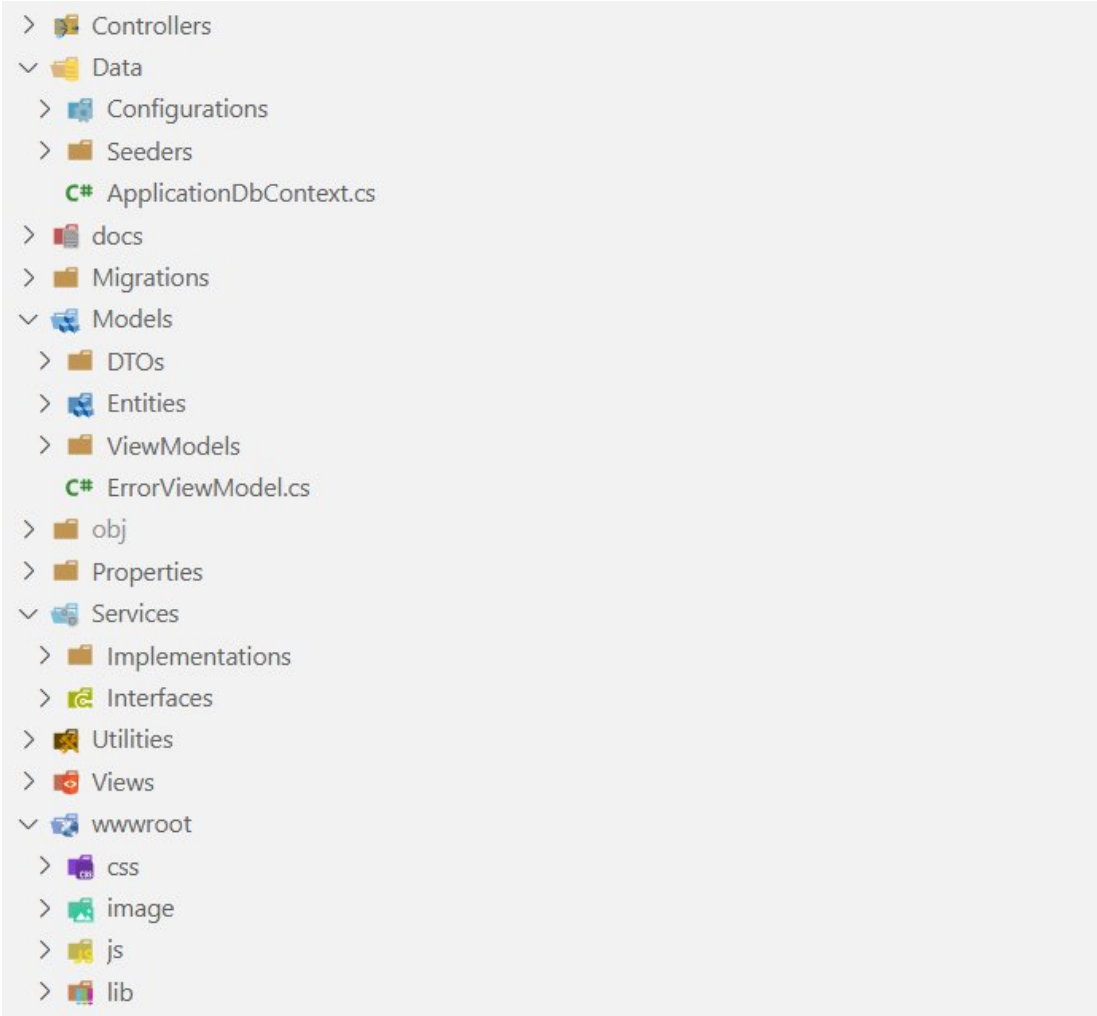
Hệ thống được thiết kế theo mô hình kiến trúc phân lớp (layered architecture), đảm bảo sự tách biệt rõ ràng về trách nhiệm giữa các thành phần. Các lớp chính bao gồm:

- Lớp trình bày (Presentation Layer): Bao gồm các Controllers và Views, chịu trách nhiệm xử lý các yêu cầu HTTP từ người dùng và hiển thị giao diện người dùng thông qua các Razor Views.
- Lớp dịch vụ (Service Layer): Đóng vai trò trung tâm trong việc xử lý logic nghiệp vụ, phối hợp giữa các thành phần và đảm bảo tính toàn vẹn của dữ liệu.
- Lớp truy cập dữ liệu (Data Access Layer): Sử dụng Entity Framework Core để tương tác với cơ sở dữ liệu, đảm bảo hiệu suất và tính nhất quán của dữ liệu.
- Lớp mô hình (Domain Model): Bao gồm các Entities và Data Transfer Objects (DTOs) để định nghĩa cấu trúc dữ liệu và truyền dữ liệu giữa các lớp.

Cấu trúc thư mục:

- /Controllers/: Chứa các controller xử lý yêu cầu HTTP.
- /Models/: Bao gồm Entities, DTOs và ViewModels để quản lý dữ liệu và giao diện.
- /Services/: Chứa các interfaces và implementations của logic nghiệp vụ.
- /Data/: Bao gồm DbContext, Configurations và Seeders để quản lý cơ sở dữ liệu.
- /Utilities/: Cung cấp các chức năng hỗ trợ như extensions và filters.
- /Views/: Chứa các Razor Views để render giao diện.
- /wwwroot/: Quản lý các tài nguyên tĩnh như CSS, JavaScript và hình ảnh.

Cấu trúc dự án:



4.3. Các quyết định thiết kế chính

4.3.1. Sử dụng mô hình MVC

- Lý do: Mô hình MVC (Model-View-Controller) được chọn vì khả năng tách biệt rõ ràng giữa logic nghiệp vụ, giao diện người dùng và xử lý yêu cầu. Điều này giúp tăng cường khả năng bảo trì và kiểm thử.
- Triển khai: Các Controllers xử lý yêu cầu người dùng, Models định nghĩa dữ liệu, và Views hiển thị giao diện người dùng.

Controller xử lý login:

```
namespace Blood_Donation_Website.Controllers
{
    [Route("account")]
    public class AccountController : Controller
    {
        private readonly IAccountService _accountService;
        public AccountController(IAccountService accountService)
        {
            _accountService = accountService;
        }

        [HttpGet("login")]
        [AllowAnonymous]
        public IActionResult Login(string? returnUrl = null)
        {
            if (User.Identity?.IsAuthenticated == true)
            {
                return RedirectToAction("Index", "Home");
            }
            ViewData["ReturnUrl"] = returnUrl;
            return View();
        }
        [HttpPost("login")]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Login(LoginViewModel model, string? returnUrl = null)
        {
            ViewData["ReturnUrl"] = returnUrl;

            if (!ModelState.IsValid)
            {
                return View(model);
            }

            try
            {
                var loginResult = await _accountService.LoginAsync(model);
                if (!loginResult)
                {
                    ModelState.AddModelError(string.Empty, "Email hoặc mật khẩu không chính xác.");
                    return View(model);
                }

                var user = await _accountService.GetUserByEmailAsync(model.Email);
                if (user == null)
                {
                    ModelState.AddModelError(string.Empty, "Có lỗi xảy ra khi đăng nhập.");
                    return View(model);
                }

                var claims = new List<Claim>
                {
                    new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
                    new Claim(ClaimTypes.Name, user.FullName),
                    new Claim(ClaimTypes.Email, user.Email),
                    new Claim(ClaimTypes.Role, user.RoleName.ToString() ?? "User")
                };

                var claimsIdentity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
                var authProperties = new AuthenticationProperties
                {
                    IsPersistent = model.RememberMe,
                    ExpiresUtc = model.RememberMe ? DateTimeOffset.UtcNow.AddDays(30) : DateTimeOffset.UtcNow.AddMinutes(30)
                };

                await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
                    new ClaimsPrincipal(claimsIdentity), authProperties);

                if (!string.IsNullOrEmpty(returnUrl) && Url.IsLocalUrl(returnUrl))
                {
                    return Redirect(returnUrl);
                }

                return RedirectToAction("Index", "Home");
            }
            catch
            {
                ModelState.AddModelError(string.Empty, "Có lỗi xảy ra khi đăng nhập. Vui lòng thử lại.");
                return View(model);
            }
        }
    }
}
```

Entity User:

```
namespace Blood_Donation_Website.Models.Entities
{
    [Table("Users")]
    public class User
    {
        [Key]
        public int UserId { get; set; }

        [Required]
        [StringLength(50)]
        public string Username { get; set; } = string.Empty;

        [Required]
        [StringLength(100)]
        public string Email { get; set; } = string.Empty;

        [Required]
        [StringLength(255)]
        public string PasswordHash { get; set; } = string.Empty;

        [Required]
        [StringLength(100)]
        public string FullName { get; set; } = string.Empty;

        [StringLength(15)]
        public string? Phone { get; set; }

        [StringLength(255)]
        public string? Address { get; set; }

        public DateTime? DateOfBirth { get; set; }

        public Gender? Gender { get; set; }

        public int? BloodTypeId { get; set; }

        public int RoleId { get; set; } = 2;

        public bool IsActive { get; set; } = true;
        public bool EmailVerified { get; set; } = false;
        public DateTime? LastDonationDate { get; set; }
        public DateTime CreatedDate { get; set; } = DateTime.Now;
        public DateTime UpdatedDate { get; set; } = DateTime.Now;

        [ForeignKey("RoleId")]
        public virtual Role Role { get; set; } = null!;
        [ForeignKey("BloodTypeId")]
        public virtual BloodType? BloodType { get; set; }

        public virtual ICollection<BloodDonationEvent> CreatedEvents { get; set; } = new List<BloodDonationEvent>();
        public virtual ICollection<DonationRegistration> DonationRegistrations { get; set; } = new List<DonationRegistration>();
        public virtual ICollection<DonationHistory> DonationHistories { get; set; } = new List<DonationHistory>();
        public virtual ICollection<HealthScreening> HealthScreenings { get; set; } = new List<HealthScreening>();
        public virtual ICollection<Notification> Notifications { get; set; } = new List<Notification>();
        public virtual ICollection<ContactMessage> ResolvedContactMessages { get; set; } = new List<ContactMessage>();
        public virtual ICollection<News> AuthoredNews { get; set; } = new List<News>();
    }
}
```

View login:

```html
<div class="auth-page">
    <div class="auth-card-container">
        <div class="auth-card">
            <div class="auth-card-body">
                <div class="auth-card-header">
                    <h1 class="auth-card-title">Chào Mừng Trở Lại!</h1>
                    <p class="auth-card-subtitle">Đăng nhập để sử dụng tất cả tính năng của trang web</p>
                </div>

                <!— Social Login Options —>
                <div class="social-login">
                    <a href="#" class="social-btn">
                        <i class="fab fa-facebook-f"></i>
                    </a>
                    <a href="#" class="social-btn">
                        <i class="fab fa-google"></i>
                    </a>
                    <a href="#" class="social-btn">
                        <i class="fab fa-linkedin-in"></i>
                    </a>
                    <a href="#" class="social-btn">
                        <i class="fab fa-github"></i>
                    </a>
                </div>

                <div class="divider">
                    <span>hoặc sử dụng email để đăng nhập</span>
                </div>

                <form asp-action="Login" asp-route-returnurl="@ViewData["ReturnUrl"]" method="post" novalidate class="compact-form">
                    <div asp-validation-summary="ModelOnly" class="alert alert-danger" role="alert"></div>

                    <div class="mb-3">
                        <input asp-for="Email" class="form-control"
                               placeholder="Địa chỉ email" autocomplete="username" />
                        <span asp-validation-for="Email" class="text-danger small"></span>
                    </div>

                    <div class="mb-3">
                        <input asp-for="Password" type="password" class="form-control"
                               placeholder="Mật khẩu" autocomplete="current-password" />
                        <span asp-validation-for="Password" class="text-danger small"></span>
                    </div>

                    <div class="d-flex justify-content-between align-items-center mb-4">
                        <div class="form-check">
                            <input asp-for="RememberMe" class="form-check-input" type="checkbox" />
                            <label asp-for="RememberMe" class="form-check-label small text-dark">
                                Ghi nhớ đăng nhập
                            </label>
                        </div>
                        <a href="#" class="text-decoration-none small" style="color: #dc3545;">
                            Quên mật khẩu?
                        </a>
                    </div>

                    <div class="d-grid">
                        <button type="submit" class="btn btn-primary">
                            ĐĂNG NHẬP
                        </button>
                    </div>
                </form>

                <div class="text-center mt-3">
                    <p class="mb-0 text-dark">
                        Chưa có tài khoản?
                        <a asp-action="Register" class="text-decoration-none fw-bold" style="color: #dc3545;">
                            Đăng ký
                        </a>
                    </p>
                </div>
            </div>
        </div>
    </div>
</div>
```

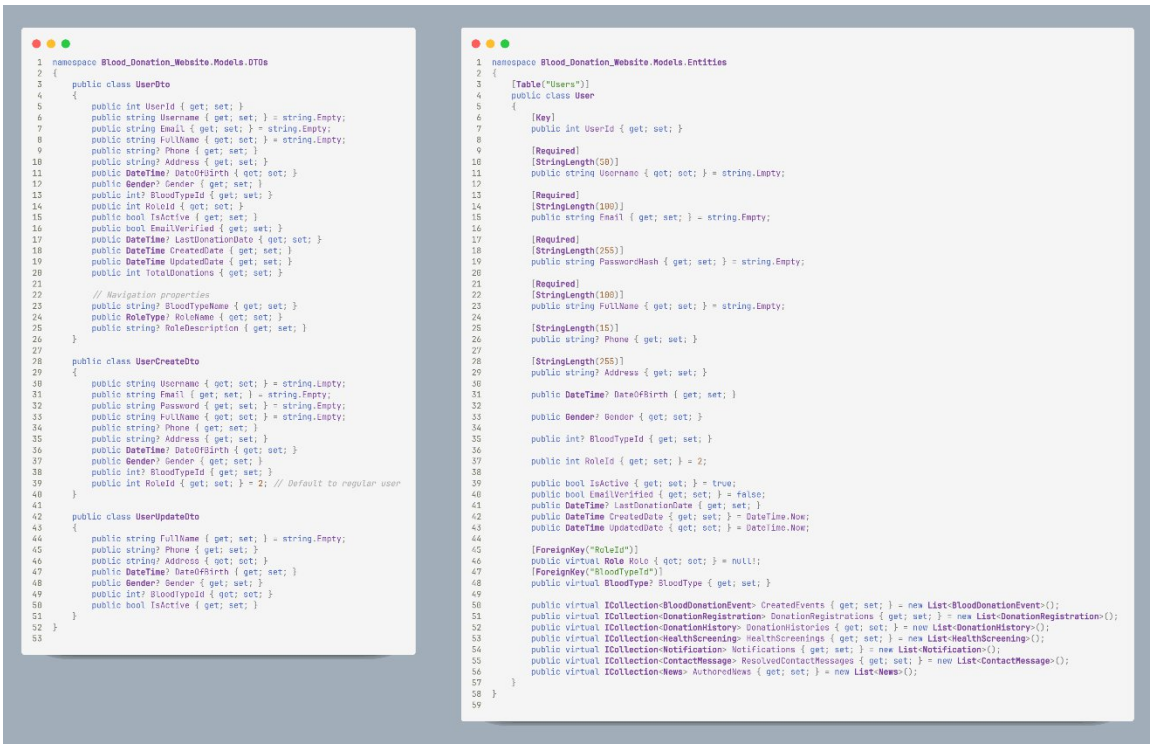
4.3.2. Entity Framework Core cho truy cập dữ liệu

- Lý do: Entity Framework Core cung cấp một cách tiếp cận mạnh mẽ và hiệu quả để ánh xạ đối tượng quan hệ (ORM), giảm thiểu mã nguồn cần viết cho các thao tác cơ sở dữ liệu.
- Triển khai: Sử dụng ApplicationDbContext để định nghĩa DbSets, cùng với các lớp Configuration để cấu hình mối quan hệ và ràng buộc dữ liệu.

4.3.3. Sử dụng DTOs và ViewModels

- Lý do: DTOs được sử dụng để tách biệt dữ liệu giữa các lớp, ngăn chặn việc lộ thông tin nhạy cảm và tối ưu hóa truyền dữ liệu. ViewModels được thiết kế để cung cấp dữ liệu được định dạng cụ thể cho giao diện người dùng.
- Triển khai: DTOs được chia thành các loại như Entity DTOs, Create/Update DTOs và Specialized DTOs. ViewModels được sử dụng để kết hợp dữ liệu từ nhiều nguồn cho các views cụ thể.

So sánh UserDto và User:



```
namespace Blood_Donation_Website.Models.DTOs
{
    public class UserDto
    {
        public int UserId { get; set; }
        public string Username { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string FullName { get; set; } = string.Empty;
        public string? Phone { get; set; }
        public string? Address { get; set; }
        public DateTime? DateOfBirth { get; set; }
        public Gender? Gender { get; set; }
        public int? BloodTypeId { get; set; }
        public int RoleId { get; set; }
        public bool IsActive { get; set; }
        public bool EmailVerified { get; set; }
        public DateTime? LastDonationDate { get; set; }
        public DateTime CreatedDate { get; set; }
        public DateTime UpdatedDate { get; set; }
        public int TotalDonations { get; set; }

        // Navigation properties
        public string? BloodTypeName { get; set; }
        public RoleType? RoleName { get; set; }
        public string? RoleDescription { get; set; }
    }

    public class UserCreateDto
    {
        public string Username { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
        public string FullName { get; set; } = string.Empty;
        public string? Phone { get; set; }
        public string? Address { get; set; }
        public DateTime? DateOfBirth { get; set; }
        public Gender? Gender { get; set; }
        public int? BloodTypeId { get; set; }
        public int RoleId { get; set; } = 2; // Default to regular user
    }

    public class UserUpdateDto
    {
        public string FullName { get; set; } = string.Empty;
        public string? Phone { get; set; }
        public string? Address { get; set; }
        public DateTime? DateOfBirth { get; set; }
        public Gender? Gender { get; set; }
        public int? BloodTypeId { get; set; }
        public bool IsActive { get; set; }
    }
}
```

```
namespace Blood_Donation_Website.Models.Entities
{
    [Table("Users")]
    public class User
    {
        [Key]
        public int UserId { get; set; }

        [Required]
        [StringLength(50)]
        public string Username { get; set; } = string.Empty;

        [Required]
        [StringLength(100)]
        public string Email { get; set; } = string.Empty;

        [Required]
        [StringLength(255)]
        public string PasswordHash { get; set; } = string.Empty;

        [Required]
        [StringLength(100)]
        public string FullName { get; set; } = string.Empty;

        [StringLength(15)]
        public string? Phone { get; set; }

        [StringLength(255)]
        public string? Address { get; set; }

        public DateTime? DateOfBirth { get; set; }

        public Gender? Gender { get; set; }

        public int? BloodTypeId { get; set; }

        public int RoleId { get; set; } = 2;

        public bool IsActive { get; set; } = true;
        public bool EmailVerified { get; set; } = false;
        public DateTime? LastDonationDate { get; set; }
        public DateTime CreatedDate { get; set; } = DateTime.Now;
        public DateTime UpdatedDate { get; set; } = DateTime.Now;

        [ForeignKey("RoleId")]
        public virtual Role Role { get; set; } = null!;
        [ForeignKey("BloodTypeId")]
        public virtual BloodType? BloodType { get; set; }

        public virtual ICollection<BloodDonationEvent> CreatedEvents { get; set; } = new List<BloodDonationEvent>();
        public virtual ICollection<DonationRegistration> DonationRegistrations { get; set; } = new List<DonationRegistration>();
        public virtual ICollection<DonationHistory> DonationHistories { get; set; } = new List<DonationHistory>();
        public virtual ICollection<HealthScreening> HealthScreenings { get; set; } = new List<HealthScreening>();
        public virtual ICollection<Notification> Notifications { get; set; } = new List<Notification>();
        public virtual ICollection<ContactMessage> ResolvedContactMessages { get; set; } = new List<ContactMessage>();
        public virtual ICollection<News> AuthoredNews { get; set; } = new List<News>();
    }
}
```

4.3.4. Dependency Injection

- Lý do: Dependency Injection (DI) được áp dụng để giảm sự phụ thuộc chặt chẽ giữa các thành phần, tăng cường khả năng kiểm thử và thay thế các triển khai.
- Triển khai: Các service interfaces được định nghĩa trong /Services/Interfaces/ và được inject vào controllers hoặc các services khác thông qua ASP.NET Core DI container.

Service xử lý tài khoản:

```
namespace Blood_Donation_Website.Services.Interfaces
{
    public interface IAccountService
    {
        /// <summary>
        /// Đăng ký tài khoản mới cho người dùng.
        /// </summary>
        Task<bool> RegisterAsync(RegisterViewModel model);
        /// <summary>
        /// Đăng nhập người dùng vào hệ thống.
        /// </summary>
        Task<bool> LoginAsync(LoginViewModel model);
        /// <summary>
        /// Đăng xuất người dùng khỏi hệ thống.
        /// </summary>
        Task<bool> LogoutAsync(string userId);
        /// <summary>
        /// Yêu cầu khôi phục mật khẩu khi người dùng quên mật khẩu.
        /// </summary>
        Task<bool> ForgotPasswordAsync(ForgotPasswordViewModel model);
        /// <summary>
        /// Đặt lại mật khẩu cho tài khoản người dùng.
        /// </summary>
        Task<bool> ResetPasswordAsync(ResetPasswordViewModel model);
        /// <summary>
        /// Thay đổi mật khẩu hiện tại của người dùng.
        /// </summary>
        Task<bool> ChangePasswordAsync(string userId, string currentPassword, string newPassword);
        /// <summary>
        /// Xác thực email của người dùng bằng token.
        /// </summary>
        Task<bool> VerifyEmailAsync(string userId, string token);
        /// <summary>
        /// Kiểm tra xem email đã tồn tại trong hệ thống chưa.
        /// </summary>
        Task<bool> IsEmailExistsAsync(string email);
        /// <summary>
        /// Kiểm tra xem người dùng có thuộc vai trò cụ thể hay không.
        /// </summary>
        Task<bool> IsUserInRoleAsync(string userId, RoleType role);
        /// <summary>
        /// Khóa tài khoản người dùng.
        /// </summary>
        Task<bool> LockUserAsync(string userId);
        /// <summary>
        /// Mở khóa tài khoản người dùng.
        /// </summary>
        Task<bool> UnlockUserAsync(string userId);
        /// <summary>
        /// Lấy thông tin người dùng theo email.
        /// </summary>
        Task<UserDto?> GetUserByEmailAsync(string email);
    }
}
```

Sử dụng services trong AccountController:

```
var loginResult = await _accountService.LoginAsync(model);
if (!loginResult)
{
    ModelState.AddModelError(string.Empty, "Email hoặc mật khẩu không chính xác.");
    return View(model);
}
```

4.3.5. Xác thực và phân quyền

- Lý do: Đảm bảo an toàn cho dữ liệu người dùng và các chức năng quản trị là yêu cầu quan trọng của hệ thống.
- Triển khai: Sử dụng ASP.NET Core Identity để quản lý xác thực và phân quyền, kết hợp với các custom filters như AdminOnlyAttribute để giới hạn truy cập.

Phân quyền cho Admin:

```
namespace Blood_Donation_Website.Utilities.Filters
{
    public class AdminOnlyAttribute : Attribute, IAuthorizationFilter
    {
        public void OnAuthorization(AuthorizationFilterContext context)
        {
            var user = context.HttpContext.User;
            if (!user.Identity?.IsAuthenticated ?? true)
            {
                context.Result = new RedirectToActionResult("Login", "Account", null);
                return;
            }

            var roleClaim = user.FindFirst(ClaimTypes.Role)?.Value;
            if (string.IsNullOrEmpty(roleClaim) || !Enum.TryParse<RoleType>(roleClaim, out var role) || role != RoleType.Admin)
            {
                context.Result = new ForbidResult();
            }
        }
    }
}
```

Phân quyền cho Doctor:

```
namespace Blood_Donation_Website.Utilities.Filters
{
    public class DoctorOnlyAttribute : Attribute, IAuthorizationFilter
    {
        public void OnAuthorization(AuthorizationFilterContext context)
        {
            var user = context.HttpContext.User;
            if (!user.Identity?.IsAuthenticated ?? true)
            {
                context.Result = new RedirectToActionResult("Login", "Account", null);
                return;
            }

            var roleClaim = user.FindFirst(ClaimTypes.Role)?.Value;
            if (string.IsNullOrEmpty(roleClaim) || !Enum.TryParse<RoleType>(roleClaim, out var role) || role != RoleType.Doctor)
            {
                context.Result = new ForbidResult();
            }
        }
    }
}
```

4.4. Các mẫu thiết kế được áp dụng

- Repository Pattern: Được triển khai thông qua Entity Framework Core DbContext, cung cấp một giao diện trừu tượng cho các thao tác cơ sở dữ liệu.
- Service Pattern: Logic nghiệp vụ được đóng gói trong các service classes, đảm bảo sự tách biệt và tái sử dụng.
- DTO Pattern: Sử dụng DTOs để truyền dữ liệu giữa các lớp, giảm sự phụ thuộc và tăng tính bảo mật.
- MVC Pattern: Được sử dụng làm nền tảng cho kiến trúc tổng thể, tách biệt rõ ràng giữa logic, dữ liệu và giao diện.
- Singleton Pattern: Được sử dụng trong một số utility services (như logging) để đảm bảo chỉ có một instance duy nhất.

4.5. Luồng dữ liệu

Luồng dữ liệu trong hệ thống được thiết kế để đảm bảo tính nhất quán và hiệu quả:

- Người dùng gửi yêu cầu HTTP đến một action trong Controller.
- Controller xác thực đầu vào và gọi phương thức phù hợp từ Service Layer.
- Service Layer thực hiện logic nghiệp vụ, tương tác với Data Access Layer khi cần.
- Data Access Layer sử dụng Entity Framework Core để truy vấn hoặc cập nhật cơ sở dữ liệu.
- Dữ liệu được trả về Controller thông qua DTOs, sau đó được truyền đến View để render HTML.
- HTML được trả về trình duyệt của người dùng để hiển thị.

5. THỰC HIỆN

5.1. Công nghệ và công cụ sử dụng

Việc phát triển *Website đăng ký hiến máu* được thực hiện dựa trên các công nghệ và công cụ sau:

- Backend: ASP.NET Core 8.0, sử dụng mô hình MVC để xây dựng kiến trúc phân lớp.
- Cơ sở dữ liệu: Entity Framework Core 8.0 làm ORM, kết hợp với SQL Server để quản lý dữ liệu.
- Frontend: Razor Views, Bootstrap 5 cho giao diện responsive, và jQuery cho các tương tác client-side.
- Xác thực và phân quyền: ASP.NET Core Identity để quản lý tài khoản người dùng và phân quyền.
- Công cụ phát triển: Visual Studio 2022, Git cho quản lý mã nguồn, và Azure DevOps, GitHub Actions cho CI/CD.
- Kiểm thử: xUnit cho unit testing và Postman cho kiểm thử API.

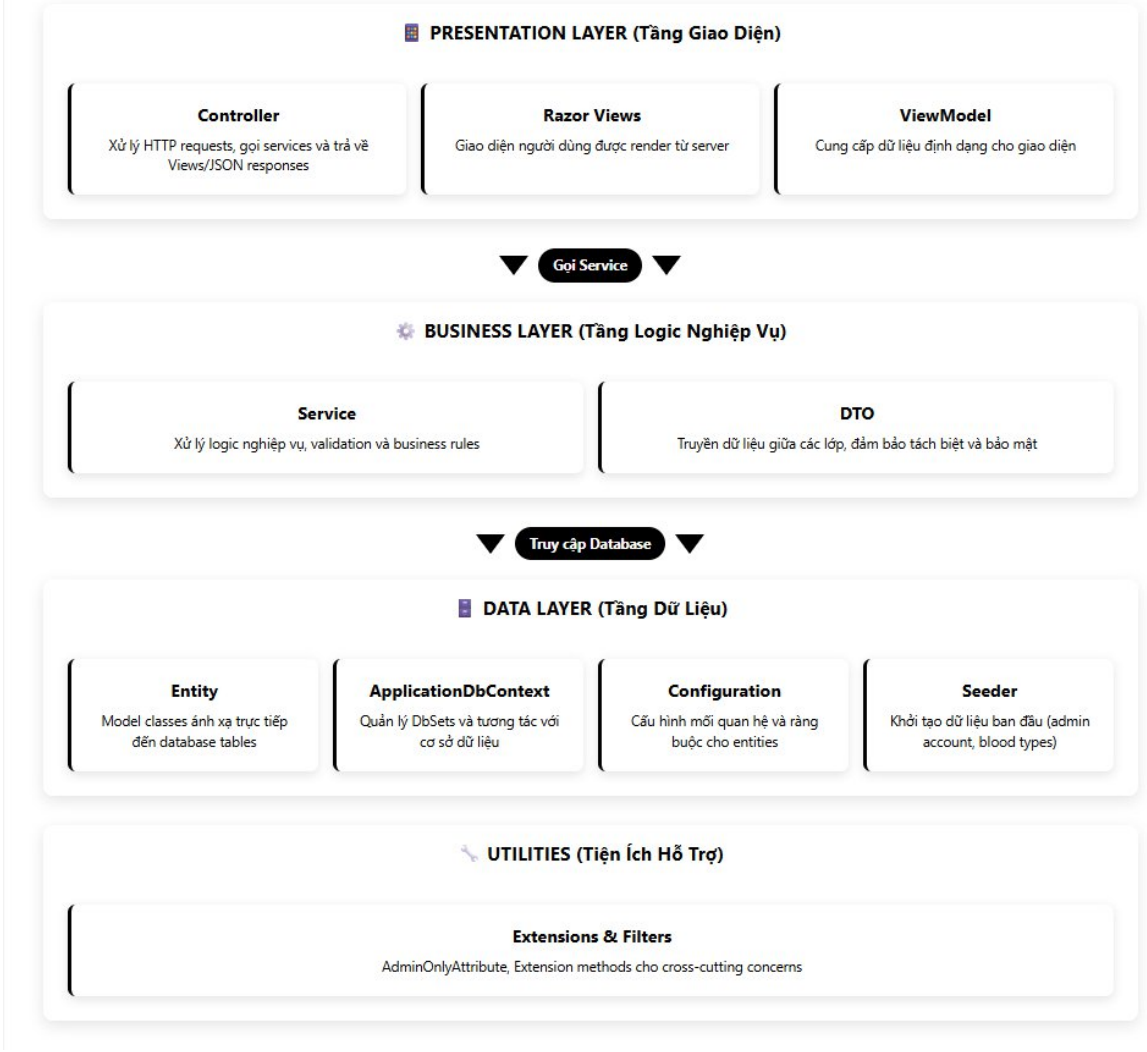
5.2. Hiện thực hóa các lớp kiến trúc

Các thành phần kiến trúc được mô tả trong tài liệu thiết kế đã được hiện thực hóa như sau:

- Controller: Mỗi controller xử lý các yêu cầu HTTP, gọi các service tương ứng và trả về Razor Views hoặc JSON responses.
- Entity: Các lớp model ánh xạ trực tiếp đến các bảng cơ sở dữ liệu thông qua Entity Framework Core.
- DTO: Các DTO truyền dữ liệu giữa các lớp, đảm bảo tách biệt và bảo mật.
- ViewModel: được xây dựng để cung cấp dữ liệu định dạng cho giao diện.
- Service: xử lý logic nghiệp vụ.
- ApplicationDbContext: quản lý các DbSets và tương tác với cơ sở dữ liệu.
- Các lớp configuration sử dụng để cấu hình mối quan hệ và ràng buộc.
- Seeder: khởi tạo dữ liệu ban đầu như tài khoản admin và danh sách nhóm máu.

- Utilities: Các phương thức mở rộng (Extention) và bộ lọc (như AdminOnlyAttribute) được triển khai để hỗ trợ các vấn đề cắt ngang như phân quyền và xử lý yêu cầu.

Sơ đồ kiến trúc ASP.NET Core MVC:



5.3. Tích hợp luồng hoạt động

5.3.1. Tổng quan

Luồng hoạt động của *Website đăng ký hiến máu* được thiết kế để quản lý quy trình hiến máu tình nguyện một cách chặt chẽ, từ đăng ký sự kiện đến hoàn tất hiến máu. Mỗi bước được ghi nhận và theo dõi thông qua các bảng cơ sở dữ liệu và các trạng thái cụ thể, đảm bảo tính minh bạch và chính xác.

5.3.2. Các bước trong quy trình hiến máu

a, Đăng ký sự kiện hiến máu

- Mô tả: Người dùng truy cập hệ thống, chọn một sự kiện hiến máu từ danh sách các sự kiện (BloodDonationEvents) và gửi yêu cầu đăng ký.
- Hành động hệ thống:
  - Tạo bản ghi trong bảng DonationRegistrations với các trường:
  - Status = 'Registered'
  - IsEligible = 0
  - Ghi nhận thông tin người dùng (UserId) và sự kiện (EventId).
- Giao diện: Người dùng sử dụng giao diện trong /Views/Account/ để đăng ký.

b, Xác nhận đăng ký (tùy chọn)

- Mô tả: Hệ thống hoặc ban tổ chức kiểm tra tính hợp lệ của đăng ký dựa trên các điều kiện như số lượng người tham gia, nhóm máu, hoặc lịch sử hiến máu.
- Hành động hệ thống:
- Nếu hợp lệ, cập nhật bản ghi trong DonationRegistrations:
  - Status = 'Confirmed'
  - IsEligible = 0
  - Nếu không hợp lệ, cập nhật:
  - Status = 'Cancelled'
  - CancellationReason với các lý do như:
  - Sự kiện đã đủ người (CurrentDonors ≥ MaxDonors).
  - Nhóm máu không phù hợp (RequiredBloodTypes).
  - Gần đây đã hiến máu (LastDonationDate quá gần).

- Tài khoản bị khóa (IsActive = 0).

- Thông báo: Gửi thông báo (Notifications.Type = 'Confirmation' hoặc 'Cancellation') qua email hoặc giao diện.

c, Check-in tại sự kiện

- Mô tả: Người dùng đến địa điểm sự kiện và thực hiện check-in trong khung giờ cho phép.
- Hành động hệ thống:
  - Cập nhật bản ghi trong DonationRegistrations:
  - Status = 'CheckedIn'
  - IsEligible = 0
  - CheckInTime = [thời gian hiện tại]
  - CheckInLocation = [địa điểm sự kiện]
  - Kiểm tra ngày và giờ sự kiện để đảm bảo check-in hợp lệ.
- Giao diện: Nhân viên sử dụng giao diện trong /Views/Admin/ để xác nhận check-in.

d, Sàng lọc sức khỏe

- Mô tả: Nhân viên y tế thực hiện kiểm tra sức khỏe và nhập thông tin vào hệ thống.
- Hành động hệ thống:
  - Tạo bản ghi trong bảng HealthScreening, liên kết với RegistrationId, bao gồm các trường:
  - Weight, Height, BloodPressure, HeartRate, Temperature, Hemoglobin, ScreeningDate.
  - Kết quả sàng lọc:
  - Đạt: Cập nhật HealthScreening.IsEligible = 1 và DonationRegistrations:
  - Status = 'Eligible'
  - IsEligible = 1
  - Không đạt: Cập nhật HealthScreening:
  - IsEligible = 0

- DisqualifyReason (ví dụ: LowHemoglobin, HighBloodPressure, RecentDonation).
- Cập nhật DonationRegistrations:
- Status = 'Ineligible'
- IsEligible = 0

- Giao diện: Trang "Chờ sàng lọc" hiển thị các đăng ký với Status = 'CheckedIn' hoặc 'Ineligible' và IsEligible = 0.

e, Hiến máu

- Mô tả: Người dùng đủ điều kiện bắt đầu quá trình hiến máu.
- Hành động hệ thống:
  - Cập nhật DonationRegistrations:
  - Status = 'Donating'
- Giao diện: Nhân viên cập nhật trạng thái qua giao diện quản trị.

f, Hoàn tất hiến máu

Mô tả: Sau khi hiến máu thành công, hệ thống ghi nhận kết quả.

Hành động hệ thống:

Tạo bản ghi trong bảng DonationHistory:

DonationDate = [thời gian hiện tại]

BloodTypeId, Volume = 350, Status = 'Completed'

NextEligibleDate = [DonationDate + thời gian nghỉ theo quy định]

Cập nhật DonationRegistrations:

Status = 'Completed'

CompletionTime = [thời gian hiện tại]

Thông báo: Gửi thông báo (Notifications.Type = 'Completion') để xác nhận hoàn tất.

Luồng hoạt động của hệ thống:

1

Đăng Ký Sự Kiện Hiến Máu

Người dùng truy cập hệ thống, chọn sự kiện từ danh sách và gửi yêu cầu đăng ký

Status: Registered

Cập nhật Database:

Bảng: DonationRegistrations
Trường: Status = 'Registered', IsEligible = 0
Ghi nhận: UserId, EventId, RegistrationDate

Ẩn chi tiết

Giao diện sử dụng:

- /Views/Account/ - Giao diện đăng ký người dùng
- Danh sách sự kiện từ BloodDonationEvents
- Form đăng ký với thông tin cá nhân

2

Xác Nhận Đăng Ký

Hệ thống kiểm tra tính hợp lệ dựa trên số lượng, nhóm máu, lịch sử hiến máu

Kiểm tra điều kiện

Hợp lệ → Confirmed

Không hợp lệ → Cancelled

Cập nhật Database:

Hợp lệ: Status = 'Confirmed', IsEligible = 0
Không hợp lệ: Status = 'Cancelled' + CancellationReason
Thông báo: Notifications.Type = 'Confirmation'/'Cancellation'

Ẩn chi tiết

Lý do hủy có thể:

- Sự kiện đã đủ người (CurrentDonors ≥ MaxDonors)
- Nhóm máu không phù hợp (RequiredBloodTypes)
- Gần đây đã hiến máu (LastDonationDate quá gần)
- Tài khoản bị khóa (IsActive = 0)

3

Check-in Tại Sự Kiện

Người dùng đến địa điểm sự kiện và thực hiện check-in trong khung giờ cho phép

Status: CheckedIn

Cập nhật Database:

Bảng: DonationRegistrations
Trường: Status = 'CheckedIn', IsEligible = 0
Ghi nhận: CheckInTime, CheckInLocation

Ẩn chi tiết

Giao diện sử dụng:

- /Views/Admin/ - Giao diện nhân viên
- Xác nhận check-in theo thời gian sự kiện
- Kiểm tra địa điểm và thời gian hợp lệ

4

Sàng Lọc Sức Khỏe

Nhân viên y tế thực hiện kiểm tra sức khỏe và nhập thông tin vào hệ thống

Kết quả sàng lọc

Đạt → Eligible

Không đạt → Ineligible

Cập nhật Database:

Bảng: HealthScreening (Weight, Height, BloodPressure, etc.)
Đạt: IsEligible = 1, Status = 'Eligible'
Không đạt: IsEligible = 0, Status = 'Ineligible' + DisqualifyReason

Ẩn chi tiết

Các chỉ số kiểm tra:

- Cân nặng, chiều cao
- Huyết áp, nhịp tim
- Nhiệt độ cơ thể
- Nồng độ Hemoglobin

Lý do loại có thể:

- LowHemoglobin, HighBloodPressure
- Fever, LowWeight, RecentDonation
- MedicalHistory, Pregnancy, etc.

5

Hiến Máu

Người dùng đủ điều kiện bắt đầu quá trình hiến máu

Status: Donating

Cập nhật Database:

Bảng: DonationRegistrations
Trường: Status = 'Donating'
Giao diện: Nhân viên cập nhật qua giao diện quản trị

Ẩn chi tiết

Quá trình hiến máu:

- Chuẩn bị dụng cụ y tế
- Tiến hành lấy máu (thường 350ml)
- Theo dõi tình trạng người hiến
- Ghi nhận thời gian bắt đầu

6

Hoàn Tất Hiến Máu

Sau khi hiến máu thành công, hệ thống ghi nhận kết quả và cập nhật lịch sử

Status: Completed

Cập nhật Database:

Bảng: DonationHistory (DonationDate, Volume = 350, Status = 'Completed')
Tính toán: NextEligibleDate = DonationDate + thời gian nghỉ
Cập nhật: DonationRegistrations.Status = 'Completed'
Thông báo: Notifications.Type = 'Completion'

Ẩn chi tiết

Hoàn tất quy trình:

- Ghi nhận thời gian hoàn thành
- Cập nhật lịch sử hiến máu
- Tính toán ngày có thể hiến tiếp theo
- Gửi thông báo xác nhận
- Cấp giấy chứng nhận (nếu có)

5.3.3. Xử lý trường hợp đặc biệt

Hủy đăng ký:

Người dùng hoặc hệ thống hủy đăng ký.

Cập nhật DonationRegistrations:

Status = 'Cancelled'

CancellationReason = [lý do cụ thể]

Gửi thông báo (Notifications.Type = 'Cancellation').

Không đến sự kiện:

Nếu người dùng không check-in trong thời gian quy định, cập nhật:

Status = 'NoShow'

Thất bại trong quá trình hiến máu:

Nếu có vấn đề xảy ra (ví dụ: vấn đề sức khỏe trong lúc hiến), cập nhật:

DonationRegistrations.Status = 'Failed'

Có thể tạo bản ghi trong DonationHistory với Status = 'Failed'

5.3.4. Bảng phân quyền

Dựa trên cơ sở dữ liệu và quy trình hiến máu, bảng phân quyền dưới đây phân định quyền truy cập và thao tác cho các vai trò (Roles: Admin, User, Hospital, Doctor, Staff).

Chức năng / Trang	Admin	User	Hospital	Doctor	Staff
Lịch sử hiến máu					
Xem lịch sử cá nhân (DonationHistory)	✓	✓			
Xem tất cả lịch sử hiến máu	✓		✓	✓	✓
Đăng ký hiến máu					
Xem sự kiện (BloodDonationEvents)	✓	✓	✓	✓	✓
Đăng ký sự kiện (DonationRegistrations)		✓			
Hủy đăng ký (Status = 'Cancelled')	✓	✓	✓	✓	✓
Xác nhận đăng ký (Status = 'Confirmed')	✓		✓	✓	✓
Check-in (Status = 'CheckedIn')	✓		✓	✓	✓
Sàng lọc sức khỏe					
Xem danh sách chờ sàng lọc			✓	✓	✓
Nhập dữ liệu sàng lọc (HealthScreening)			✓	✓	✓
Cập nhật kết quả sàng lọc (Eligible/Ineligible)			✓	✓	✓
Quản lý sự kiện hiến máu					
Tạo/sửa/xóa sự kiện (BloodDonationEvents)	✓		✓		
Cập nhật trạng thái sự kiện	✓		✓		✓
Thông báo					
Xem thông báo cá nhân (Notifications)	✓	✓	✓	✓	✓
Gửi thông báo	✓		✓	✓	✓
Hồ sơ người dùng					
Xem/cập nhật hồ sơ cá nhân (Users)	✓	✓			
Xem hồ sơ người dùng khác	✓		✓	✓	
Xác minh email (EmailVerified)	✓	✓			
Báo cáo và thống kê					
Xem báo cáo (DonationHistory, HealthScreening)	✓		✓	✓	✓

Xuất báo cáo	✓		✓		
Quản lý danh mục tin tức					
Tạo/sửa/xóa danh mục (NewsCategories)	✓				
Quản lý tin tức					
Tạo/sửa/xóa tin tức (News)	✓		✓		✓
Phê duyệt tin tức (IsPublished)	✓		✓		
Quản lý liên hệ					
Xem/xử lý tin nhắn (ContactMessages)	✓		✓		✓
Quản lý cài đặt hệ thống					
Cập nhật cài đặt (Settings)	✓				

Admin: Quyền toàn diện, quản lý toàn bộ hệ thống.

User: Người dùng thông thường, chỉ thao tác trên dữ liệu cá nhân (đăng ký, xem lịch sử, hồ sơ).

Hospital: Quản lý sự kiện, báo cáo, và xử lý dữ liệu liên quan đến hiến máu.

Doctor: Thực hiện sàng lọc sức khỏe, xem dữ liệu người dùng và báo cáo.

Staff: Hỗ trợ check-in, nhập dữ liệu sàng lọc, quản lý sự kiện cơ bản.

6. KIỂM THỬ

6.1. Tổng quan

6.1.1. Thông tin chung

Hệ thống đăng ký hiến máu nhân đạo hỗ trợ người hiến máu đăng ký tài khoản, xem và tham gia sự kiện hiến máu, cũng như theo dõi quy trình hiến máu từ check-in, sàng lọc đến hoàn tất. Người dùng có thể cập nhật thông tin cá nhân (họ tên, email, số điện thoại, địa chỉ, nhóm máu,…) trên trang cá nhân. Sau khi đăng ký và đăng nhập thành công hệ thống cấp mã định danh và gửi email xác nhận đăng ký.

Người hiến máu có thể xem danh sách các sự kiện hiến máu sắp diễn ra (tên sự kiện, ngày/giờ, địa điểm, mô tả) và tìm kiếm theo địa điểm. Tại chi tiết sự kiện, người dùng bấm "Đăng ký tham gia" để tạo đăng ký. Hệ thống kiểm tra điều kiện (như khoảng cách lần hiến trước, kết quả sức khỏe) và nếu đủ điều kiện sẽ tạo bản ghi mới trong bảng DonationRegistrations với trạng thái ban đầu "Registered" (đang chờ); nếu không đủ điều kiện, hệ thống thông báo lý do và không tạo đăng ký. Người dùng có thể hủy đăng ký trước ngày sự kiện.

Khi đến ngày diễn ra sự kiện, người hiến thực hiện check-in; hệ thống đối chiếu đăng ký và cập nhật trạng thái đăng ký là 'CheckedIn' và lưu thời gian check-in. Tiếp theo, nhân viên y tế tiến hành sàng lọc sức khỏe: các thông số (cân nặng, huyết áp, huyết sắc tố,…) được ghi vào bảng HealthScreening. Nếu đủ điều kiện, hệ thống gán IsEligible = 1 và cập nhật trạng thái là 'Eligible' cho DonationRegistrations; nếu không đủ điều kiện, gán trạng thái là 'Ineligible' và lưu nguyên nhân loại. Cuối cùng, nếu hiến máu thành công, một bản ghi mới được thêm vào bảng DonationHistory ghi nhận ngày giờ, thể tích, nhóm máu, và trạng thái đăng ký là 'Completed'; đồng thời cập nhật trạng thái lịch sử hiến là 'Completed' và thời gian hoàn tất. Người dùng có thể xem lại lịch sử hiến máu (DonationHistory) và thông báo cá nhân (bảng Notifications) trên giao diện.

Ngoài ra, hệ thống có phân quyền cho quản trị viên (Admin) để quản lý tài khoản người dùng, tạo và quản lý sự kiện, đăng tin tức/văn bản, phê duyệt kết quả sàng lọc, cũng như thống kê và cấu hình hệ thống.

6.1.2. Lý do kiểm thử

- Trong bối cảnh thực tế, hệ thống đăng ký và quản lý hiến máu nhân đạo đóng vai trò quan trọng trong việc đảm bảo vận hành hiệu quả các chiến dịch hiến máu, kết nối người hiến với đơn vị tổ chức và đảm bảo an toàn cho người tham gia.
- Việc kiểm thử hệ thống giúp phát hiện và loại bỏ các lỗi tiềm ẩn trước khi triển khai chính thức, từ đó đảm bảo hệ thống hoạt động ổn định, dữ liệu chính xác và trải nghiệm người dùng mượt mà.
- Kiểm thử còn giúp xác nhận các quy trình nghiệp vụ tuân thủ đúng quy định chuyên môn y tế (như sàng lọc, tính thời gian hiến lại), từ đó giảm thiểu rủi ro và tăng độ tin cậy.

6.1.3. Vai trò kiểm thử

- Phát hiện lỗi sớm: Giảm chi phí sửa lỗi bằng cách phát hiện lỗi trong giai đoạn phát triển thay vì sau triển khai.
- Đảm bảo chất lượng: Kiểm soát và cải tiến chất lượng phần mềm trước khi đưa vào sử dụng thực tế.
- Tăng độ tin cậy: Đảm bảo hệ thống vận hành ổn định, đáng tin cậy khi xử lý dữ liệu y tế và lịch sử hiến máu.
- Cải thiện trải nghiệm người dùng: Giảm tình trạng lỗi giao diện, thao tác khó hiểu, hoặc luồng sử dụng bị gián đoạn.
- Giảm rủi ro vận hành: Tránh tình trạng dữ liệu sai lệch, quá tải sự kiện, hoặc cho phép người không đủ điều kiện tham gia.
- Đáp ứng đúng yêu cầu nghiệp vụ: Đảm bảo các chức năng tuân thủ quy định chuyên môn, đặc biệt trong môi trường nhạy cảm như y tế.

6.1.4. Các vấn đề phát sinh nếu không kiểm thử

Nếu hệ thống không được kiểm thử kỹ lưỡng, có thể phát sinh nhiều vấn đề nghiêm trọng như:

- Người dùng không đăng ký hoặc đăng nhập được do lỗi xác thực.
- Dữ liệu hiến máu sai lệch, dẫn đến tính sai lịch hiến tiếp theo.
- Hệ thống cho phép người không đủ điều kiện tham gia, gây rủi ro y tế.
- Sự kiện bị đăng ký vượt quá số lượng cho phép, gây quá tải thực tế.
- Thiếu thông báo hoặc thông báo sai thời điểm, làm người dùng bỏ lỡ thông tin quan trọng.
- Giao diện lỗi, trải nghiệm người dùng kém, ảnh hưởng đến uy tín tổ chức.

Những lỗi này nếu không được phát hiện sớm có thể làm gián đoạn quy trình hiến máu, ảnh hưởng đến sức khỏe người tham gia, làm sai lệch dữ liệu y tế và ảnh hưởng đến uy tín của hệ thống.

6.2. Mục tiêu kiểm thử

Giai đoạn kiểm thử của Website đăng ký hiến máu nhằm đảm bảo rằng hệ thống đáp ứng đầy đủ các yêu cầu nghiệp vụ, hoạt động chính xác theo luồng hoạt động đã thiết kế, và đáp ứng các tiêu chuẩn bảo mật và hiệu suất. Cụ thể, kiểm thử tập trung vào:

- Xác thực chức năng chính: Đảm bảo các chức năng đăng ký tài khoản, đăng nhập, cập nhật thông tin cá nhân, xem sự kiện, đăng ký/hủy sự kiện và quy trình hiến máu (check-in, sàng lọc, hiến máu, ghi nhận lịch sử) hoạt động đúng theo đặc tả.
- Kiểm thử tính toàn vẹn dữ liệu: Kiểm tra các bản ghi dữ liệu được lưu và cập nhật chính xác trong các bảng Users, BloodDonationEvents, DonationRegistrations, HealthScreening, DonationHistory, v.v. đảm bảo tuân thủ quy trình nghiệp vụ. Ví dụ, đăng ký sự kiện phải tạo bản ghi tương ứng trong DonationRegistrations; thông số sàng lọc được lưu trong HealthScreening; kết thúc hiến máu tạo bản ghi trong DonationHistory.
- Kiểm thử giao diện và xử lý lỗi: Đánh giá giao diện người dùng (gồm biểu mẫu đăng ký, đăng nhập, form nhập liệu) hoạt động đúng; hiển thị thông báo lỗi phù hợp khi đầu vào không hợp lệ (thiếu thông tin bắt buộc, định dạng sai, mật khẩu không khớp, v.v.).
- Kiểm thử điều kiện biên và kịch bản đặc biệt: Thực hiện các kịch bản tiêu cực và biên ví dụ như thiếu trường thông tin, đăng ký trùng lặp, đăng ký khi sự kiện đã đủ người, check-in sai thời gian, sàng lọc thất bại (ví dụ huyết áp quá cao, huyết sắc tố thấp, v.v. dẫn đến Ineligible). Đảm bảo hệ thống xử lý đúng mọi tình huống đặc biệt (hủy đăng ký, không đến sự kiện, huỷ quy trình hiến máu, v.v.).
- Đánh giá độ ổn định: Xác định hệ thống không có lỗi nghiêm trọng trong phạm vi kiểm thử, đảm bảo chất lượng phần mềm tốt, sẵn sàng triển khai vào môi trường thực tế.

6.3. Phương pháp kiểm thử

6.3.1. Loại kiểm thử

- UI Testing: Kiểm tra giao diện người dùng trên các trình duyệt (Chrome, Firefox) để đảm bảo tính tương thích và trải nghiệm người dùng. Ví dụ: kiểm tra form đăng ký và thông báo lỗi.
- Integration Testing: Kiểm tra tích hợp giữa các thành phần như Controllers, Services, và ApplicationDbContext để đảm bảo luồng dữ liệu hoạt động đúng.
- Unit Testing: Kiểm tra các đơn vị mã nguồn riêng lẻ, ví dụ: logic kiểm tra điều kiện đăng ký trong IBloodDonationEventService.

6.3.2. Công cụ kiểm thử

- Môi trường phát triển: .NET Framework
- Selenium: Kiểm tra giao diện người dùng tự động trên nhiều trình duyệt.
- Postman: Kiểm tra các API endpoints (nếu có) để xác nhận phản hồi JSON.
- Appium: Kiểm thử ứng dụng di động (iOS, Android).
- JMeter: Kiểm thử hiệu năng.

6.3.3. Môi trường kiểm thử

- Local: Môi trường phát triển trên máy tính cá nhân với SQL Server Express.
- Staging: Môi trường SQL Database để kiểm tra tích hợp trước khi triển khai.
- Hệ điều hành và phần cứng: Thực hiện kiểm thử trên môi trường Windows 10 (64-bit) với cấu hình phần cứng chuẩn (CPU Intel i5, RAM 8GB).
- Trình duyệt: Sử dụng trình duyệt web Chrome (phiên bản mới nhất) và Firefox để kiểm thử giao diện, đảm bảo hỗ trợ trên nhiều trình duyệt phổ biến.

6.4. Test Case

Các trường hợp kiểm thử (Test Case) được chia thành các nhóm chính, tương ứng với các bước trong luồng hoạt động hiến máu và chức năng đăng nhập/đăng ký tài khoản. Dưới đây là tóm tắt các nhóm và một số trường hợp tiêu biểu:

6.4.1. Đăng ký tài khoản

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
REG01	Đăng ký thành công với thông tin hợp lệ	1. Truy cập trang đăng ký.	Họ tên: Nguyễn Văn A	Đăng ký thành công, chuyển trang xác thực email
		2. Nhập đầy đủ thông tin hợp lệ.	Email: test@gmail.com	
		3. Click "Đăng ký".	SĐT: 0123456789	
			Mật khẩu: Test123@	
REG02	Validate email trùng lặp	1. Nhập email đã tồn tại	Email: test@gmail.com	Hiển thị lỗi "Email đã được sử dụng"
		2. Nhập thông tin khác hợp lệ	(email đã tồn tại)	
		3. Click "Đăng ký"		
REG03	Validate trường bắt buộc - Họ tên	1. Để trống họ tên	Họ tên: (trống)	Hiển thị lỗi "Họ tên là bắt buộc"
		2. Nhập thông tin khác hợp lệ	Email: test2@gmail.com	
		3. Click "Đăng ký"	Các trường khác hợp lệ	
REG04	Validate format email không hợp lệ	1. Nhập email sai format	Email: invalid-email	Hiển thị lỗi "Email không đúng định dạng"
		2. Nhập thông tin khác hợp lệ	Email: test@	
		3. Click "Đăng ký"	Email: @gmail.com	
REG05	Validate số điện thoại không hợp lệ	1. Nhập SĐT sai format	SĐT: 123 (quá ngắn)	Hiển thị lỗi "Số điện thoại không hợp lệ"
		2. Nhập thông tin khác hợp lệ	SĐT: abcxyz	
		3. Click "Đăng ký"	SĐT: 012345678901 (quá dài)	
REG06	Validate xác nhận mật khẩu không khớp	1. Nhập mật khẩu	Password: Test123@	Hiển thị lỗi "Mật khẩu xác nhận không khớp"
		2. Nhập xác nhận mật khẩu khác	Confirm: Test456@	
		3. Click "Đăng ký"		

6.4.2. Đăng nhập

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
LG01	Đăng nhập thành công với thông tin hợp lệ	1. Truy cập trang Đăng nhập	Email: test@gmail.com	Đăng nhập thành công, chuyển đến trang chủ người dùng
		2. Nhập đúng email/mật khẩu	Mật khẩu: Test123@	
		3. Nhấn "Đăng nhập"		
LG02	Email chưa đăng ký	1. Nhập email chưa đăng ký	Email: notexist@gmail.com	Hiển thị lỗi "Email không tồn tại"
		2. Nhập mật khẩu bất kỳ	Mật khẩu: abc123	
		3. Click "Đăng nhập"		
LG03	Mật khẩu không đúng	1. Nhập email đúng	Email: test@gmail.com	Hiển thị lỗi "Mật khẩu không chính xác"
		2. Nhập mật khẩu sai	Mật khẩu: sai123	
		3. Click "Đăng nhập"		
LG04	Email sai định dạng	1. Nhập email không đúng định dạng	Email: abc.com	Hiển thị lỗi "Email không đúng định dạng"
		2. Nhập mật khẩu bất kỳ	Email: @gmail.com	
		3. Click "Đăng nhập"	Mật khẩu: Test123@	
LG05	Bỏ trống mật khẩu	1. Nhập email hợp lệ	Email: test@gmail.com	Hiển thị lỗi "Vui lòng nhập mật khẩu"
		2. Để trống mật khẩu	Mật khẩu: (trống)	
		3. Click "Đăng nhập"		
LG06	Bỏ trống email	1. Để trống email	Email: (trống)	Hiển thị lỗi "Vui lòng nhập email"
		2. Nhập mật khẩu hợp lệ	Mật khẩu: Test123@	
		3. Click "Đăng nhập"		

6.4.3. Đăng ký sự kiện

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
DK01	Đăng ký hiến máu thành công với thông tin hợp lệ.	Nhập thông tin vào form đăng ký; nhấn **Đăng ký**.	Thông tin người dùng (họ tên, SĐT hợp lệ) và ID sự kiện còn chỗ.	Hệ thống lưu đăng ký, trạng thái cập nhật 'Registered'; hiển thị thông báo thành công.
DK02	Đăng ký thất bại khi thiếu thông tin bắt buộc (ví dụ: không điền SĐT).	Nhập form đăng ký thiếu thông tin SĐT; nhấn **Đăng ký**.	Bỏ trống trường SĐT.	Hệ thống hiển thị lỗi nhập liệu và không cho phép đăng ký.
DK03	Đăng ký thất bại khi đã đăng ký trước đó (trùng lặp).	Nhập thông tin đăng ký cho cùng sự kiện lần thứ hai.	Người dùng đã có đăng ký cho sự kiện này.	Hệ thống hiển thị cảnh báo đã đăng ký và không tạo bản ghi mới.
DK04	Đăng ký thất bại khi sự kiện đã đủ số lượng hiến máu viên.	Thực hiện đăng ký bình thường.	Sự kiện có số lượng hiến viên hiện tại = số tối đa.	Hệ thống thông báo sự kiện đã đầy, không cho đăng ký thêm.
DK05	Hủy đăng ký trước ngày hiến máu (luồng đặc biệt).	Người dùng chọn hủy đăng ký.	Người dùng đã đăng ký thành công, sự kiện chưa diễn ra.	Trạng thái đăng ký được cập nhật thành 'Cancelled'; lý do (nếu có) được lưu.
DK06	Đăng ký không thành công khi đã hết hạn đăng ký hoặc sự kiện đã kết thúc.	Thử thực hiện đăng ký.	Thời điểm hiện tại > thời hạn đăng ký hoặc sau ngày sự kiện.	Hệ thống thông báo không thể đăng ký (đã đóng đăng ký).

6.4.4. Check-in sự kiện

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
CI01	Check-in thành công khi người hiến đến đúng thời gian của sự kiện.	Thực hiện check-in tại điểm hiến máu.	Người dùng đã đăng ký hợp lệ, ngày hiện tại trùng ngày sự kiện.	Trạng thái đăng ký cập nhật thành 'CheckedIn'; hiển thị thông báo thành công.
CI02	Check-in thất bại khi người hiến chưa đăng ký.	Thực hiện check-in.	Người dùng chưa đăng ký tham gia sự kiện.	Hệ thống hiển thị lỗi (đăng ký không tồn tại).
CI03	Check-in thất bại khi sự kiện đã kết thúc.	Thực hiện check-in sau khi sự kiện kết thúc.	Ngày hiện tại > ngày kết thúc sự kiện.	Hệ thống từ chối và thông báo sự kiện không khả dụng.
CI04	No-show: người đăng ký không đến điểm hiến và không check-in.	Sau khi sự kiện kết thúc, hệ thống xử lý tự động.	Người dùng đã đăng ký nhưng không thực hiện check-in khi sự kiện kết thúc.	Trạng thái 'NoShow' được gán cho đăng ký; thông báo No-show.
CI05	Hủy sau khi đã check-in trước khi sàng lọc (luồng đặc biệt).	Người dùng hoặc nhân viên hủy quy trình ngay sau khi check-in.	Người dùng đã thực hiện check-in nhưng chưa sàng lọc.	Trạng thái đăng ký cập nhật thành 'Cancelled'; ngừng quy trình.

6.4.5. Sàng lọc sức khoẻ

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
SL01	Sàng lọc thành công khi các chỉ số sức khỏe đều hợp lệ.	Bác sĩ nhập các chỉ số và lưu kết quả.	Cân nặng ≥45kg, huyết áp và nhịp tim bình thường, Hb≥120g/l, nhiệt độ ≤37°C.	Đánh giá đủ điều kiện (IsEligible=true); trạng thái 'Eligible'.
SL02	Sàng lọc thất bại do huyết sắc tố dưới chuẩn (<120g/l).	Nhập Hb và lưu kết quả.	Hb = 110g/l (<120).	Đánh giá không đủ điều kiện; trạng thái 'Ineligible'; lý do LowHemoglobin.
SL03	Sàng lọc thất bại do huyết áp cao (ví dụ 160/100 mmHg).	Nhập huyết áp và lưu kết quả.	Huyết áp = 160/100 mmHg.	Đánh giá không đủ điều kiện; trạng thái 'Ineligible'; lý do HighBloodPressure.
SL04	Sàng lọc thất bại do cân nặng dưới 45kg.	Nhập cân nặng và lưu.	Cân nặng = 40kg (<45kg).	Đánh giá không đủ điều kiện; trạng thái 'Ineligible'; lý do LowWeight.
SL05	Sàng lọc thất bại do thân nhiệt cao (sốt >37.5°C).	Đo nhiệt độ và lưu.	Nhiệt độ = 38°C (>37.5°C).	Đánh giá không đủ điều kiện; trạng thái 'Ineligible'; lý do Fever.

6.4.6. Hiến máu

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
HM01	Hiến máu thành công khi hoàn tất lấy đủ thể tích quy định.	Thực hiện hiến máu đến đủ thể tích.	Đã sàng lọc đủ điều kiện, bắt đầu hiến máu.	Cập nhật trạng thái 'Completed'; cấp giấy chứng nhận; lưu thể tích hiến.
HM02	Quá trình hiến máu dừng giữa chừng do sự cố y tế (người hiến ngất).	Nhân viên ngừng quy trình sau khi người hiến ngất.	Đã bắt đầu lấy máu; người hiến ngất sau vài phút.	Trạng thái 'Failed' hoặc 'Stopped'; lưu lý do; không cấp giấy chứng nhận.
HM03	Hiến máu thất bại do lỗi thiết bị (kim, máy bơm).	Hủy bỏ quy trình hiến máu do lỗi kỹ thuật.	Bắt đầu hiến máu nhưng xảy ra lỗi thiết bị.	Trạng thái 'Failed'; thông báo lỗi thiết bị.
HM04	Người hiến dừng hiến máu giữa chừng chủ ý (rút lui).	Dừng quy trình theo yêu cầu người hiến.	Đang hiến máu, người hiến yêu cầu dừng.	Trạng thái 'Cancelled'; lưu lý do.

6.4.7. Hoàn tất

ID	Mô tả	Bước thực hiện	Dữ liệu Test	Kết quả mong đợi
HT01	Hoàn tất quy trình hiến máu thành công.	Hệ thống cấp giấy chứng nhận và gửi thông báo.	Đã hiến máu thành công (trạng thái 'Completed').	Người hiến nhận giấy chứng nhận; trạng thái 'Completed'; thông báo được gửi.
HT02	Người hiến không nhận giấy chứng nhận nếu hiến máu không hoàn thành.	Kết thúc quy trình.	Trạng thái hiện 'Failed' hoặc 'Ineligible'.	Không cấp giấy chứng nhận; trạng thái vẫn 'Failed' hoặc 'Cancelled'.
HT03	Ngày hiến máu tiếp theo được tính chính xác sau lần hiến thành công.	Hệ thống tính toán ngày hiến tiếp theo.	Hiến máu thành công ngày 01-01-2025.	Ngày hiến tiếp theo = 01-04-2025 (sau 12 tuần); cập nhật NextEligibleDate.
HT04	Cập nhật thông báo xác nhận hiến máu thành công đến người dùng.	Hệ thống gửi email/SMS xác nhận.	Đăng ký có hiến máu thành công.	Người dùng nhận được email/SMS cảm ơn và xác nhận.

6.5. Kiểm thử bảo mật và RBAC

Mục tiêu: Đảm bảo rằng chỉ người dùng có vai trò phù hợp (Admin, Hospital, Doctor, Staff, User) mới truy cập được các chức năng cụ thể (được mô tả trong Bảng phân quyền).

Trường hợp kiểm thử:

- Kiểm tra rằng người dùng không có vai trò Admin không thể truy cập các action trong AdminController (ví dụ: xác nhận đăng ký hoặc quản lý sự kiện).
- Kiểm tra rằng nhân viên có thể nhập dữ liệu sàng lọc sức khỏe (HealthScreening) nhưng không thể chỉnh sửa sự kiện (BloodDonationEvents).
- Phương pháp: Sử dụng unit test cho AdminOnlyAttribute và integration test để xác minh các endpoint trả về lỗi 403 (Forbidden) khi truy cập không được phép.

Kết quả: Tất cả các test case liên quan đến RBAC đều đạt trạng thái PASS, đảm bảo tính an toàn của hệ thống.

6.6. Thực hiện kiểm thử

- Kiểm thử chức năng (Functional Testing): Dựa trên đặc tả chức năng và luồng nghiệp vụ xây dựng và thực thi các test case bao phủ các tính năng chính. Các bước kiểm thử được ghi trong báo cáo, bao gồm các kịch bản tiêu chuẩn và tiêu cực cho từng chức năng (đăng ký sự kiện, check-in, sàng lọc, hiến máu, đăng ký tài khoản, đăng nhập).
- Kiểm thử tích hợp (Integration Testing): Kiểm tra sự tương tác giữa các module và cơ sở dữ liệu. Ví dụ, xác nhận rằng sau khi check-in, hệ thống cập nhật đúng trạng thái và thời gian tại DonationRegistrations; sau khi hiến máu hoàn tất, hệ thống tạo bản ghi trong DonationHistory và cập nhật Status='Completed'. Đã có các test case tương ứng trong TestCase.xlsx để xác minh các cập nhật này.
- Kiểm thử giao diện (UI Testing): Đánh giá các form đăng ký, đăng nhập, nhập liệu (validate email, mật khẩu, thông báo lỗi) tương thích trên các trình duyệt khác nhau. Ví dụ, các test case kiểm tra định dạng email, mật khẩu, các trường bắt buộc để đảm bảo thông báo lỗi hiển thị đúng và không cho phép ghi sai thông tin.
- Kiểm thử đơn vị (Unit Testing): Kiểm thử các chức năng xử lý logic nhỏ độc lập. Trong trường hợp này, có test đơn vị tính toán NextEligibleDate sau lần hiến máu dựa trên ngày hiến gần nhất và quy định thời gian nghỉ. Test case này đã xác nhận công thức tính đúng thời gian sau 12 tuần đối với nữ hoặc 3 tháng đối với nam.
- Kiểm thử kịch bản đặc biệt: Thực hiện các kịch bản như hủy đăng ký trước/sau check-in, người đăng ký không đến (No-show), hủy trong khi hiến máu do sự cố y tế hoặc ý muốn cá nhân (cập nhật Status='Cancelled' hoặc Failed tương ứng). Kiểm thử những tình huống này đảm bảo hệ thống xử lý đúng quy trình đóng gói dữ liệu theo quy định trong flow đặc biệt.
- Phương pháp thiết kế test case: Sử dụng kỹ thuật phân vùng tương đương và giá trị biên cho các trường dữ liệu (ví dụ ngày, email, số điện thoại). Các test case được triển khai thủ công dựa trên file TestCase.xlsx (phân nhóm theo chức năng) và ghi nhận kết quả, ghi chú rõ loại test (UI, Integration, Unit).

6.7. Kết quả kiểm thử

- Tổng số test case: 36
- Kết quả: Tất cả các test case được liệt kê trong tài liệu đều đạt trạng thái PASS, chứng minh rằng hệ thống hoạt động đúng theo yêu cầu.
- Vấn đề phát sinh: Không có lỗi nghiêm trọng được ghi nhận. Một số lỗi giao diện nhỏ (như căn chỉnh form đăng ký) đã được khắc phục trong giai đoạn phát triển.

6.8. Thách thức và giải pháp

- Thách thức: Kiểm thử các trường hợp đặc biệt như "No-show" (CI-04) hoặc hiến máu thất bại (HM-02) yêu cầu mô phỏng dữ liệu phức tạp.
- Giải pháp: Sử dụng dữ liệu seed từ /Data/Seeders/ để tạo các bản ghi mẫu trong DonationRegistrations và DonationHistory, sau đó chạy kiểm thử tự động để xác minh trạng thái.
- Thách thức: Đảm bảo tính đồng bộ khi nhiều người dùng đăng ký cùng lúc (DK-04).
- Giải pháp: Thêm kiểm thử tích hợp với cơ chế khóa (locking) trong SQL Server để ngăn chặn đăng ký trùng lặp.

6.9. Kết luận

- Giai đoạn kiểm thử đã xác nhận rằng *Website đăng ký hiến máu* đáp ứng đầy đủ các yêu cầu nghiệp vụ và luồng hoạt động được mô tả trong luồng hoạt dộng.
- Các test case bao phủ toàn bộ các chức năng chính (đăng ký, check-in, sàng lọc, hiến máu, đăng nhập/đăng ký tài khoản) và đảm bảo rằng hệ thống hoạt động ổn định, an toàn, và thân thiện với người dùng.
- Kết quả kiểm thử cho thấy hệ thống sẵn sàng để triển khai vào môi trường thực tế.

7. TRIỂN KHAI

7.1. Mục tiêu triển khai

Hệ thống Website đăng ký hiến máu được phát triển trên nền tảng ASP.NET Core, sử dụng SQL Server làm cơ sở dữ liệu, mã nguồn quản lý trên GitHub. Mục tiêu triển khai là đảm bảo hệ thống hoạt động ổn định, bảo mật, dễ bảo trì và sẵn sàng mở rộng.

Github: https://github.com/minhngocgithub/Blood-Donation

7.2. Quản Lý Mã Nguồn & Quy Trình CI/CD

Repository: Mã nguồn lưu trữ trên GitHub, tổ chức theo các thư mục: Controllers, Models, Services, Data, Utilities, Views, wwwroot.

Chiến lược branch:

- main: Mã nguồn ổn định, sẵn sàng triển khai.
- develop: Tích hợp các tính năng mới, kiểm thử trước khi merge vào main.
- feature/, bugfix/: Phát triển tính năng hoặc sửa lỗi, merge qua pull request.

Pull Request & Code Review: Sử dụng pull request để kiểm tra mã nguồn, review trước khi hợp nhất.

Tích hợp kiểm thử tự động: Sử dụng GitHub Actions để tự động build và chạy unit test/integration test khi có thay đổi mã nguồn.

Quản lý issue: Theo dõi lỗi và yêu cầu tính năng mới qua GitHub Issues.

7.3. Chuẩn Bị Môi Trường Phát Triển

Yêu cầu hệ thống:

.NET SDK 8.0 trở lên

SQL Server Express/Developer Edition

Visual Studio 2022 hoặc IDE tương thích (Rider, VS Code...)

Cấu hình kết nối cơ sở dữ liệu:

Mở file appsettings.json hoặc appsettings.Development.json ở thư mục gốc.

Cập nhật giá trị DefaultConnection theo SQL Serve, ví dụ:

Thay localhost\\SQLEXPRESS bằng tên server nếu khác.

Nếu dùng SQL Authentication, thay bằng User ID=...;Password=....

Chạy ở chế độ Development:

Visual Studio: Mặc định chạy ở môi trường Development.

Command line: Đặt biến môi trường:

PowerShell: $env:ASPNETCORE_ENVIRONMENT = "Development"

CMD: set ASPNETCORE_ENVIRONMENT=Development

Linux/macOS: export ASPNETCORE_ENVIRONMENT=Development

7.4. Khởi Tạo & Cập Nhật Cơ Sở Dữ Liệu

Tạo migration ban đầu: add-migration InitialCreate

Áp dụng migration để tạo schema: update-database

Khởi tạo dữ liệu mẫu:

```
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

    context.Database.EnsureCreated();

    context.SeedData();
}
```

Các Seeder trong /Data/Seeders/ sẽ tự động được gọi trong Program.cs để tạo tài khoản admin, nhóm máu, v.v.

7.5. Build, Kiểm Thử & Đóng Gói

Build project:

Chạy kiểm thử:

Đóng gói (nếu cần):

Có thể sử dụng Dockerfile (nếu có) để build image container.

7.6. Cấu Hình Triển Khai

- Biến môi trường: Định nghĩa các biến nhạy cảm (API key, connection string) qua môi trường hoặc hệ thống quản lý bí mật (Azure Key Vault, AWS Secrets Manager).
- Docker: Nếu triển khai container, sử dụng Dockerfile để build image.
- Cloud: Khuyến nghị triển khai trên Azure App Service, AWS Elastic Beanstalk hoặc máy chủ on-premises.

- Cơ sở dữ liệu: Sử dụng Azure SQL Database, Amazon RDS hoặc SQL Server riêng.
- Email & Logging: Tích hợp dịch vụ email (SendGrid) và logging (Azure Application Insights, Serilog...).

7.7. Bảo Mật & Hiệu Suất

- Bảo mật:
  - Sử dụng ASP.NET Core Identity, mã hóa dữ liệu người dùng.
  - RBAC với các custom filter (AdminOnlyAttribute, ...).
  - Bảo vệ CSRF, XSS qua tính năng mặc định của ASP.NET Core.
- Hiệu suất:
  - Tối ưu truy vấn, sử dụng index (cấu hình trong /Data/Configurations/).
  - Sử dụng in-memory caching cho dữ liệu truy vấn thường xuyên.

7.8. Quản Lý & Bảo Trì

- Cập nhật định kỳ: Theo dõi và cập nhật các thư viện, package.
- Tài liệu: Đọc documents để biết hướng dẫn cài đặt, chạy, kiểm thử.
- Cộng tác: Đóng góp qua fork, pull request, hoặc báo cáo issue trên GitHub.